

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA BAZE DE DATE

LUCRARE DE DISERTAȚIE

Batch vs Streaming în Sisteme de Date

Conducător științific
conf. dr. Dan Mircea Suciu

Absolvent
Hognogi Ana-Maria Cristina

2026

**BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION DATABASES**

DISSERTATION THESIS

Batch vs Streaming in Data Systems

Supervisor
assoc. prof. Dan Mircea Suciu PhD

Author
Hognogi Ana-Maria Cristina

2026

ABSTRACT

This thesis addresses the need for efficient data processing models in modern applications, where large volumes of data are continuously generated and must be processed reliably. The work focuses on two fundamental paradigms: batch processing and stream processing. To make the comparison easier to understand, the thesis uses a hospital monitoring scenario, where patient vitals, alerts, medical activities, and analytical reports highlight the differences between the two approaches.

The main objective of this work is to analyze the trade-offs between batch and stream processing. Batch processing is suitable for scheduled analysis over larger datasets, such as medical statistics, patient evolution, and retrospective treatment analysis, but it introduces higher latency. Stream processing enables immediate handling of incoming events, such as detecting abnormal patient vitals in real time, but it requires a more complex architecture and continuous system operation.

The original contribution of this thesis consists of the design and implementation of MedStream, a demonstrative system based on simulated hospital data. The application integrates both batch and streaming pipelines within a unified architecture and presents the results through a visual web interface. It is not intended as a real clinical tool but as an educational and experimental platform for understanding how the two paradigms behave under similar conditions.

The thesis is organized into several chapters. **Chapter 1** introduces the research topic, outlining the motivation, objectives, and original contribution of the study. **Chapter 2** presents the theoretical foundations related to batch processing, stream processing, and relevant architectural models. **Chapter 3** describes the technologies used in the development of MedStream and their role in the application. **Chapter 4** presents the implementation of MedStream, including the application architecture, database model, processing flows, simulator, and main backend and frontend components. **Chapter 5** presents the use of the application and the analysis of the generated results, including the comparison between streaming and batch processing. Finally, **Chapter 6** concludes the thesis and discusses the limitations of the system and possible future improvements.

Hognogi Ana-Maria Cristina



Cuprins

1	Introducere	1
1.1	Motivația și obiectivele lucrării	1
1.2	Contribuția originală	2
1.3	Structura lucrării	2
2	Procesarea datelor și modele arhitecturale	4
2.1	Fundamente ale procesării datelor	4
2.1.1	Procesarea datelor în sisteme moderne	4
2.1.2	Procesarea de tip batch	5
2.1.2.1	Caracteristici	6
2.1.2.2	Avantaje și limitări	6
2.1.3	Procesarea de tip streaming	7
2.1.3.1	Timpul în streaming	8
2.1.3.2	Avantaje și limitări	9
2.2	Modele arhitecturale și domenii de aplicare	9
2.2.1	Soluții și arhitecturi existente	9
2.2.1.1	Apache Kafka	9
2.2.1.2	AWS Glue Jobs	11
2.2.1.3	Arhitectura Lambda	12
2.2.1.4	Motoare moderne de procesare	13
2.2.2	Criterii de comparație	13
2.2.3	Exemple de utilizare	14
2.2.4	Moduri actuale de explicare	16
3	Tehnologii utilizate pentru aplicație	17
3.1	Tehnologii pentru backend	17
3.1.1	Python	17
3.1.2	Virtual environments	18
3.1.3	FastAPI și REST API	18
3.1.4	Pydantic	19
3.1.5	CRUD operations	19

3.1.6	CORS middleware	19
3.2	Tehnologii pentru frontend	20
3.2.1	JavaScript	20
3.2.2	React	20
3.2.3	Single Page Application	21
3.2.4	JSX	21
3.2.5	React Hooks	21
3.2.6	Design system și stilizare	21
3.2.7	Recharts	22
3.3	Tehnologii pentru comunicarea dintre componente	22
3.3.1	HTTP și API-uri REST	22
3.3.2	WebSocket	23
3.3.3	Securitate, validare și integritatea datelor	24
3.4	Tehnologii pentru persistența datelor	24
3.4.1	PostgreSQL	24
3.4.2	SQLAlchemy ORM	25
3.5	Procesarea evenimentelor și a datelor	25
3.5.1	Apache Kafka	25
3.5.2	APScheduler	26
3.5.3	Apache Spark	26
3.6	Containerizare și rulare locală	27
3.7	Integrarea tehnologiilor	28
4	Implementarea aplicației MedStream	30
4.1	Prezentare generală și arhitectură	30
4.1.1	Cazuri de utilizare	32
4.1.2	Fluxul de procesare a alertelor	33
4.1.3	Modelul bazei de date	34
4.2	Implementarea sistemului	37
4.2.1	Configurarea mediului de rulare	37
4.2.1.1	Configurarea serverului	37
4.2.1.2	Configurarea bazei de date	38
4.2.1.3	Configurarea frontend-ului	38
4.2.2	Implementarea backend	38
4.2.2.1	Sistemul de autentificare	38
4.2.2.2	Rute API	39
4.2.2.3	Validarea datelor și repository-uri	39
4.2.2.4	Servicii streaming și WebSocket	39
4.2.2.5	Controller-ul de procesare batch	40

4.2.3	Implementarea frontend	40
4.2.3.1	Rutare și pagini protejate	40
4.2.3.2	Navigation bar și layout	41
4.2.3.3	Serviciul API și clientul WebSocket	41
4.2.3.4	Componente reutilizabile și grafice	41
4.2.3.5	Pagini clinice	41
4.2.4	Instrumente auxiliare pentru dezvoltare	42
4.2.4.1	Monitorizarea topicurilor Kafka	42
4.2.4.2	Testarea fluxurilor de email	42
4.3	Simulatorul de date și integrarea fluxurilor	43
4.3.1	Rolul simulatorului	43
4.3.2	Generarea valorilor vitale și a alertelor	43
4.3.3	Pregătirea datelor pentru analiza tratamentului	44
4.3.4	Integrarea cu procesarea streaming	44
4.3.5	Integrarea cu procesarea batch	44
5	Scenarii de utilizare și rezultatele obținute	46
5.1	Rezultatele generate și metricile aplicației	46
5.1.1	Metrici generale	46
5.1.2	Latența streaming și batch	47
5.1.3	Evoluția alertelor	48
5.1.4	Interpretarea rezultatelor	49
5.2	Raportarea la studii experimentale	49
5.2.1	Compararea cu rezultate din surse existente	50
5.3	Funcționalități	51
5.3.1	Funcționalități utilizator	51
5.3.1.1	Autentificare și profil doctor	51
5.3.1.2	Managementul pacienților	53
5.3.2	Funcționalități generate de sistem	56
5.3.2.1	Dashboard și alerte	56
5.3.2.2	Metrici streaming	57
5.3.2.3	Metrici batch	57
5.3.2.4	Comparație streaming și batch	57
5.3.2.5	Analiza tratamentului	58
6	Concluzii	62
	Bibliografie	64

Capitolul 1

Introducere

Diferența dintre procesarea datelor în timp real și procesarea periodică este importantă în multe aplicații moderne, mai ales atunci când același sistem trebuie să ofere atât reacții rapide, cât și rezultate agregate. Lucrarea pornește de la această diferență și propune o aplicație demonstrativă prin care procesarea streaming și procesarea batch pot fi observate în același scenariu funcțional.

1.1 Motivația și obiectivele lucrării

În multe aplicații actuale, datele nu apar toate deodată, ci sunt generate constant de utilizatori, senzori, servicii sau sisteme externe. Din acest motiv, modul în care aceste date sunt procesate influențează direct cât de repede poate reacționa o aplicație și ce tip de informații poate oferi. Unele date trebuie tratate imediat, de exemplu pentru afișarea unei alerte sau actualizarea unui dashboard, în timp ce alte date pot fi analizate la anumite intervale, atunci când este nevoie de statistici, rapoarte sau interpretări pe o perioadă mai mare.

Această lucrare pornește de la diferența dintre procesarea de tip streaming și procesarea de tip batch. Deși cele două concepte sunt întâlnite frecvent în sistemele de date, diferența dintre ele poate deveni greu de urmărit atunci când este explicată doar prin definiții sau prin exemple strict tehnice. Din acest motiv, am ales să construiesc o aplicație demonstrativă, numită MedStream, prin care cele două abordări pot fi observate în același scenariu.

Scenariul folosit este inspirat din monitorizarea medicală. Într-un astfel de sistem pot apărea continuu date despre pacienți, valori vitale, alerte și activități medicale. Aceste date sunt potrivite pentru a evidenția atât nevoia de reacție rapidă, specifică procesării streaming, cât și nevoia de analiză periodică, specifică procesării batch. Obiectivul principal al lucrării este compararea celor două modele de procesare printr-o aplicație care include monitorizarea live și generarea de alerte.

1.2 Contribuția originală

Contribuția originală a lucrării constă în proiectarea și implementarea aplicației MedStream. Aplicația integrează într-un singur sistem atât un flux de procesare streaming, cât și un flux de procesare batch, folosind același tip de date simulate. Scopul aplicației nu este acela de a înlocui un sistem medical real, ci de a oferi un exemplu practic prin care diferențele dintre cele două abordări pot fi înțelese mai ușor.

Componenta de streaming este folosită pentru monitorizarea valorilor vitale ale pacienților și pentru generarea alertelor atunci când anumite praguri sunt depășite. Componenta de batch este folosită pentru calcule periodice pe baza datelor acumulate, cum ar fi statistici despre pacienți, departamente sau activități medicale. Prin folosirea aceluiași scenariu pentru ambele fluxuri, aplicația permite o comparație mai clară între procesarea imediată a evenimentelor și procesarea periodică a datelor deja colectate.

O altă parte importantă a contribuției este interfața web a aplicației, care prezintă rezultatele într-o formă vizuală. Prin dashboard-uri, tabele, grafice și alerte, utilizatorul poate observa mai ușor ce informații sunt disponibile în timp real și ce informații sunt obținute după executarea proceselor batch.

1.3 Structura lucrării

Lucrarea este organizată în șase capitole, urmărind trecerea de la contextul teoretic al procesării datelor la proiectarea, implementarea și evaluarea aplicației MedStream. Structura este construită astfel încât diferența dintre procesarea streaming și procesarea batch să fie analizată mai întâi conceptual, apoi demonstrată practic printr-un sistem funcțional.

Primul capitol introduce tema lucrării și stabilește cadrul general al cercetării. Sunt prezentate motivația alegerii subiectului, obiectivele urmărite și contribuția originală a aplicației MedStream. Capitolul explică de ce comparația dintre procesarea batch și procesarea streaming este relevantă în aplicații moderne și de ce un scenariu de monitorizare clinică simulată este potrivit pentru evidențierea diferenței dintre cele două abordări.

Capitolul al doilea prezintă fundamentele teoretice ale procesării datelor, cu accent pe procesarea batch și procesarea streaming. Sunt discutate concepte precum date limitate și nelimitate, latență, actualitatea rezultatelor, corectitudine, complexitate și cost operațional. De asemenea, capitolul include modele și tehnologii relevante pentru domeniu, precum Apache Kafka, AWS Glue, Arhitectura Lambda și motoare moderne de procesare, pentru a contura contextul în care cele două paradigme sunt utilizate.

Capitolul al treilea descrie tehnologiile utilizate în dezvoltarea aplicației MedStream și rolul acestora în cadrul sistemului. Sunt prezentate tehnologiile pentru backend, frontend, comunicarea dintre componente, persistența datelor, procesarea evenimentelor și rularea locală a aplicației. Accentul este pus pe justificarea alegerilor tehnice și pe modul în care aceste tehnologii permit separarea responsabilităților dintre interfață, logică aplicațională, bază de date, mesagerie și procesare periodică.

Capitolul al patrulea este dedicat implementării aplicației MedStream. Sunt prezentate arhitectura concretă a sistemului, cazurile de utilizare, fluxul de procesare a alertelor, modelul bazei de date și configurarea mediului de rulare. Capitolul descrie și implementarea componentelor backend și frontend, serviciile de streaming, mecanismul batch, simulatorul de date clinice sintetice și modul în care fluxurile de procesare sunt integrate în cadrul aplicației.

Capitolul al cincilea prezintă utilizarea aplicației și analiza rezultatelor obținute în timpul rulării. Sunt descrise funcționalitățile disponibile în interfață, procesele automate generate de sistem, metricile streaming și batch, precum și comparația dintre cele două modele de procesare. Rezultatele sunt interpretate în raport cu scopul demonstrativ al aplicației și cu observațiile din literatura de specialitate privind latența, actualizarea datelor și analiza periodică.

Capitolul al șaselea sintetizează concluziile lucrării. Sunt discutate rezultatele obținute, limitele sistemului implementat și posibile direcții de dezvoltare viitoare. Concluziile evidențiază modul în care MedStream demonstrează complementaritatea dintre procesarea streaming și procesarea batch într-o aplicație hibridă, construită în jurul aceluiași scenariu și al acelorași tipuri de date.

Această organizare permite delimitarea treptată a conceptelor teoretice, justificarea alegerilor tehnologice și analiza soluției implementate. Înainte de prezentarea arhitecturii și a implementării aplicației, este necesară clarificarea modelelor de procesare batch și streaming, precum și a contextului arhitectural în care acestea sunt utilizate.

Pe parcursul redactării acestei lucrări, inteligența artificială generativă a fost utilizată pentru îmbunătățirea formulării textului și pentru redesenarea unor elemente vizuale, oferind sprijin în organizarea diagramelor și în rafinarea frazelor. Prin îmbunătățirea clarității, fără a influența direcția conținutului, această tehnologie a avut un rol de suport în procesul de redactare.

Capitolul 2

Procesarea datelor și modele arhitecturale

Compararea procesării batch cu procesarea streaming presupune clarificarea modului în care datele sunt colectate, delimitate, procesate și transformate în rezultate utile. În funcție de momentul procesării și de tipul datelor analizate, cele două abordări răspund unor nevoi diferite: reacție rapidă la evenimente recente sau analiză periodică asupra datelor acumulate.

2.1 Fundamente ale procesării datelor

2.1.1 Procesarea datelor în sisteme moderne

În aplicații care lucrează cu date generate constant, cum sunt sistemele de monitorizare, nu este suficient ca informațiile să fie doar stocate. Momentul în care datele sunt procesate influențează direct modul în care sistemul poate reacționa. O alertă medicală are valoare doar dacă este produsă rapid, în timp ce un raport statistic poate fi calculat periodic, pe baza datelor acumulate [Dat26].

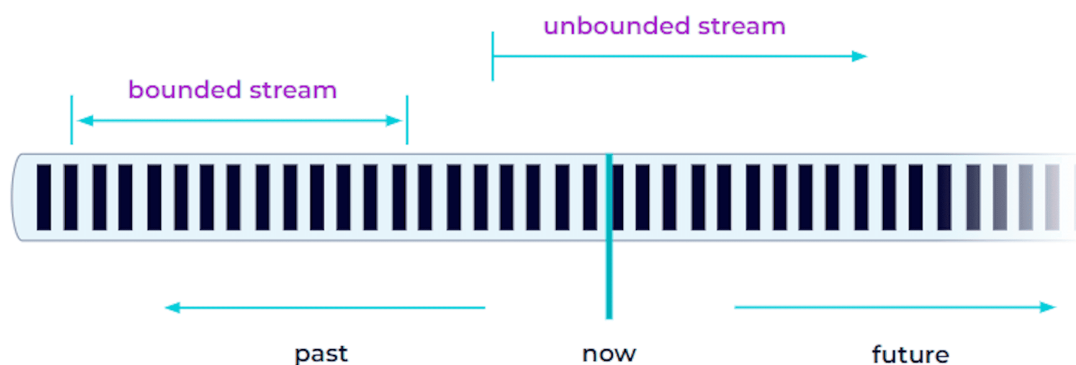


Figura 2.1: Diferența dintre un flux limitat și un flux nelimitat [Con26].

Procesarea datelor presupune transformarea informațiilor brute în rezultate utile. Înainte ca datele să fie afișate într-o interfață sau utilizate pentru luarea unor decizii, ele pot fi colectate, validate, filtrate, agregate și analizate. Modul în care aceste etape sunt organizate influențează direct latența, complexitatea, consumul de resurse și tipul de scenarii pentru care sistemul este potrivit [ABC⁺15].

În procesarea datelor, se face distincția între date limitate și date nelimitate. Datele limitate au un început și un sfârșit clar, iar datele nelimitate apar continuu și nu au un final prestabilit. Figura 2.1 ilustrează această diferență și ajută la înțelegerea modului în care batch processing și stream processing tratează datele.

Din perspectivă arhitecturală, sistemele moderne trebuie să gestioneze simultan volum, viteză și varietate. Volumul se referă la cantitatea de date procesată, viteza la ritmul în care datele apar și trebuie tratate, iar varietatea la formele diferite pe care datele le pot avea. Batch-ul răspunde mai bine nevoilor de analiză asupra volumelor mari, iar streaming-ul răspunde mai bine nevoilor de reacție rapidă la evenimente [K⁺19].

2.1.2 Procesarea de tip batch

Procesarea de tip batch reprezintă o abordare în care datele sunt colectate pentru o anumită perioadă de timp și apoi sunt procesate împreună. În loc ca fiecare eveniment să fie tratat imediat, sistemul așteaptă acumularea unui set de date, după care execută operații precum filtrare, transformare, agregare sau generare de rapoarte [Ama26e].

Un exemplu simplu este generarea unui raport zilnic. Pe parcursul zilei, sistemul colectează date, iar la finalul zilei calculează statistici, totaluri sau indicatori de performanță [Dat26]. În acest caz, faptul că rezultatul nu este disponibil imediat nu reprezintă o problemă, deoarece scopul este analiza unei perioade complete, nu reacția rapidă la un eveniment individual.

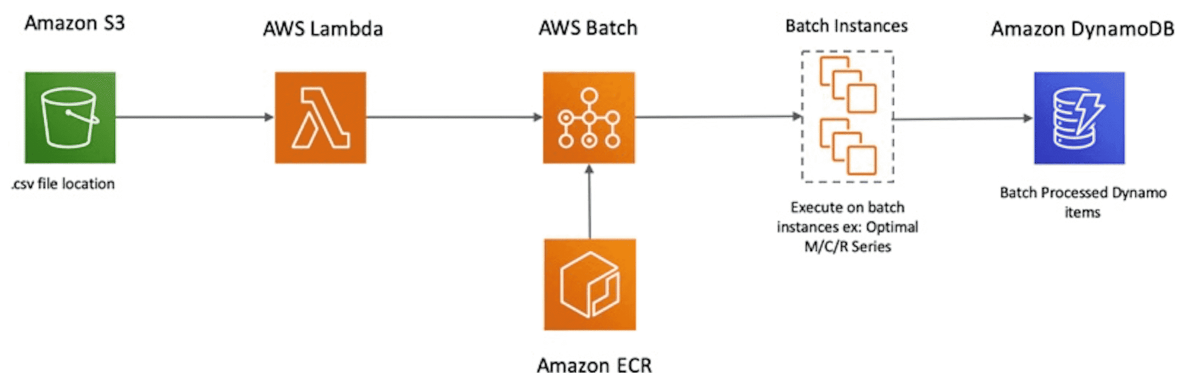


Figura 2.2: Proces batch orchestrat cu AWS Batch [Ram19].

Procesarea batch este folosită frecvent în sisteme în care datele sunt declanșate de fișiere, joburi programate sau sarcini de calcul care pot fi executate separat de interacțiunea directă cu utilizatorul. Figura 2.2 prezintă un exemplu de flux batch în care un fișier din Amazon S3 declanșează un proces prin AWS Lambda, iar AWS Batch execută jobul într-un mediu de calcul dedicat.

Procesarea batch a fost utilizată mult timp în sisteme informatice și rămâne importantă în aplicații moderne. Un exemplu relevant în acest domeniu este modelul MapReduce, propus de Dean și Ghemawat, care a introdus un mod de procesare a volumelor mari de date pe clustere de calcul [DG08].

Modelul MapReduce a influențat puternic procesarea batch deoarece a separat logica aplicației de detaliile execuției distribuite. Dezvoltatorul definește operații de tip map și reduce, iar sistemul se ocupă de împărțirea lucrului, execuția paralelă, gestionarea erorilor și combinarea rezultatelor. Această idee este importantă deoarece arată cum un set mare de date poate fi prelucrat prin pași simpli, dar executați la scară mare [DG08].

2.1.2.1 Caracteristici

O caracteristică importantă a procesării batch este faptul că datele sunt tratate ca un set finit. Sistemul cunoaște, cel puțin la momentul execuției, datele care trebuie procesate. Acest lucru permite aplicarea unor operații globale, cum ar fi sortarea completă, agregarea tuturor valorilor sau recalcularea unor rezultate pe baza întregului set de date [DG08].

Un alt aspect important este repetabilitatea. Dacă un proces batch produce un rezultat greșit, acesta poate fi reluat pe același set de date, după corectarea logicii de procesare [ABC⁺15]. Această proprietate este utilă în scenarii în care corectitudinea rezultatelor este mai importantă decât viteza cu care acestea sunt obținute.

Procesarea batch este de obicei executată la intervale stabilite. Aceste intervale pot fi de câteva minute, câteva ore, o zi sau chiar o perioadă mai lungă, în funcție de nevoile aplicației [Ama26e]. Prin urmare, batch-ul oferă o imagine stabilă asupra datelor acumulate, dar nu oferă întotdeauna o imagine actualizată în timp real.

2.1.2.2 Avantaje și limitări

Un avantaj major al procesării batch este simplitatea conceptuală. Deoarece datele sunt disponibile înainte de procesare, sistemul poate fi proiectat mai ușor, iar rezultatele pot fi verificate mai clar. În plus, batch-ul este potrivit pentru calcule complexe, rapoarte periodice, analize istorice și procesări care nu necesită răspuns imediat [Ama26e].

Principala limitare este latența. Rezultatele devin disponibile doar după ce datele au fost colectate și după ce procesul batch a fost executat. În aplicațiile în care evenimentele trebuie tratate rapid, această întârziere poate reduce utilitatea informației [Dat26].

O altă limitare apare atunci când volumul de date este foarte mare, iar procesele batch durează mult. Într-un astfel de caz, rezultatele pot fi deja depășite în momentul în care devin disponibile. Din acest motiv, batch processing este potrivit pentru analiză și raportare, dar nu este suficient pentru toate scenariile moderne de procesare a datelor [ABC⁺15].

2.1.3 Procesarea de tip streaming

Procesarea de tip streaming presupune tratarea datelor pe măsură ce acestea apar. În loc ca sistemul să aștepte acumularea unui set complet de date, evenimentele sunt procesate continuu, individual sau în grupuri foarte mici. Această abordare este potrivită pentru situațiile în care informația are valoare imediată și în care sistemul trebuie să reacționeze rapid [SÇZ05].

Un eveniment poate reprezenta o acțiune a unui utilizator, o tranzacție, o măsurătoare de la un senzor, o eroare într-un sistem sau orice modificare relevantă pentru aplicație. În procesarea streaming, aceste evenimente formează un flux continuu, iar sistemul citește fluxul, aplică transformări, actualizează stări interne și produce rezultate noi pe măsură ce datele ajung [Apa26b].

Un mod obișnuit de a reprezenta streaming-ul este printr-o topologie de procesare, în care datele intră printr-un procesor sursă, sunt transformate de procesoare intermediare și ajung la un procesor de ieșire. Figura 2.3 ilustrează această idee printr-o topologie Kafka Streams, unde nodurile reprezintă pași de procesare, iar muchiile reprezintă fluxurile de date dintre aceștia.

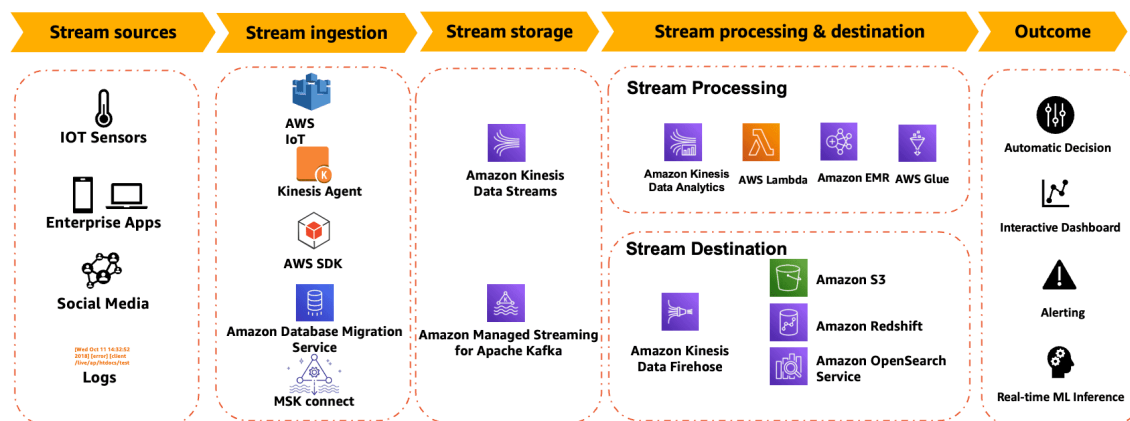


Figura 2.3: Arhitectură modernă pentru procesarea streaming [Ama26d].

Event streaming este folosit în multe sisteme moderne deoarece permite captarea, stocarea și procesarea evenimentelor într-un mod continuu. În documentațiile actuale, streaming-ul este asociat cu procesare cu latență redusă, monitorizare live, reacții automate și fluxuri de evenimente care pot continua pe termen nelimitat.

Față de batch, streaming-ul pornește de la ideea că fluxul de date rămâne deschis și că rezultatele trebuie actualizate pe măsură ce apar evenimente noi. Fluxul este considerat deschis, iar sistemul trebuie să producă rezultate parțiale, actualizabile și uneori aproximative. Această abordare este utilă atunci când valoarea informației scade rapid dacă este procesată prea târziu [SCZ05].

Sistemele de stream processing trebuie să rezolve probleme care nu apar la fel de puternic în batch, precum evenimente întârziate, ordine diferită a mesajelor, menținerea stării și recuperarea după erori fără pierderea consistenței. Din acest motiv, streaming-ul nu trebuie înțeles doar ca procesare rapidă, ci ca un model de procesare continuă care implică decizii arhitecturale mai complexe [ABC+15].

2.1.3.1 Timpul în streaming

O problemă importantă în procesarea streaming este modul în care sistemul interpretează timpul. Într-un caz ideal, evenimentele ar ajunge în sistem exact în ordinea în care au fost produse. În practică, evenimentele pot întârzia, pot ajunge în altă ordine sau pot proveni din surse diferite, fiecare cu propriul ritm de generare [ABC+15].

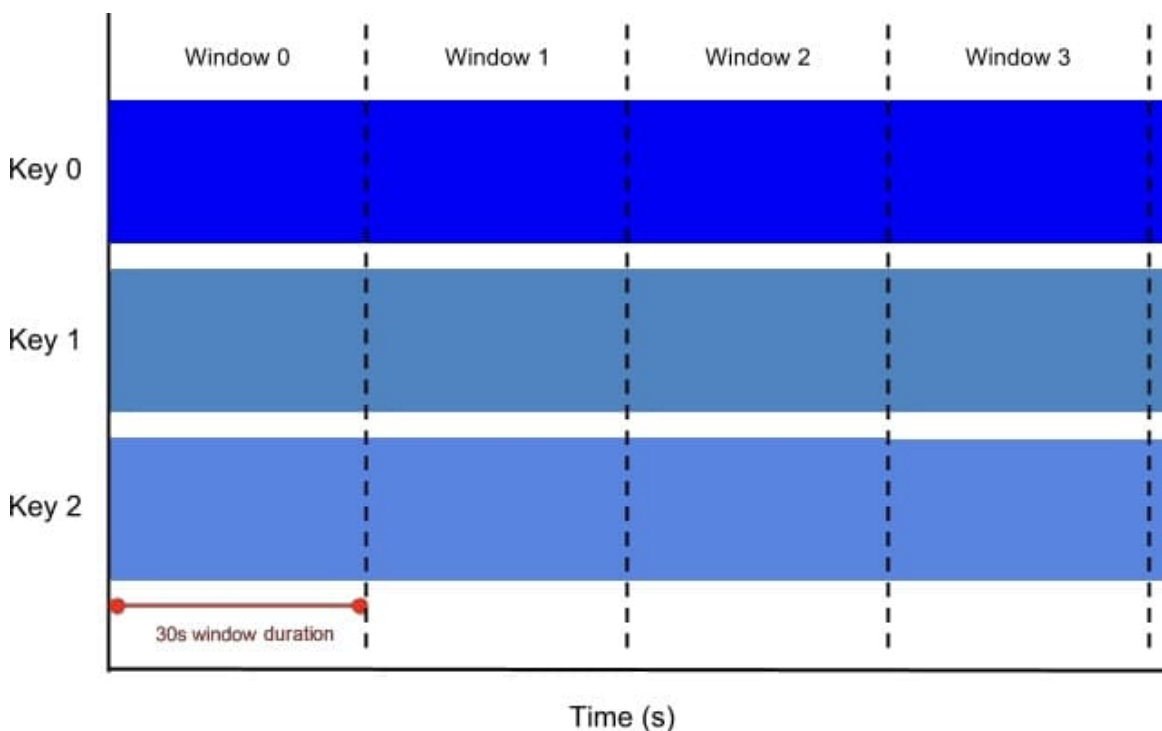


Figura 2.4: Exemplu de ferestre de timp fixe în procesarea streaming [Apa26a].

Din acest motiv, se face diferența între timpul evenimentului și timpul procesării. Timpul evenimentului reprezintă momentul în care evenimentul s-a produs în realitate, iar timpul procesării reprezintă momentul în care evenimentul este primit și tratat de sistem. Această distincție este importantă pentru agregări, alerte și calcule care depind de intervale temporale [ABC⁺15].

Pentru a lucra cu fluxuri continue, sistemele de streaming folosesc ferestre de timp. O fereastră grupează evenimentele produse într-un anumit interval, de exemplu ultimele treizeci de secunde sau ultimele cinci minute. Figura 2.4 prezintă un exemplu de ferestre fixe, unde evenimentele sunt încadrate în intervale temporale constante [Apa26a].

2.1.3.2 Avantaje și limitări

Principalul avantaj al procesării streaming este latența redusă. Rezultatele pot fi obținute aproape imediat după apariția evenimentelor, ceea ce permite reacții rapide. Această caracteristică este importantă în aplicații de monitorizare, detectare a fraudelor, analiză a comportamentului utilizatorilor, sisteme IoT sau dashboard-uri live [SCZ05].

Un alt avantaj este actualitatea datelor. Într-un sistem streaming, interfața sau componentele consumatoare pot primi informații noi fără să aștepte următoarea execuție periodică. Acest lucru permite observarea schimbărilor într-un timp scurt și declanșarea unor acțiuni automate atunci când apar condiții importante [Apa26b].

Limitarea principală este complexitatea. Un sistem streaming rulează continuu, gestionează evenimente întârziate, tratează erori și păstrează stare pentru anumite calcule. Corectitudinea rezultatelor devine mai dificil de garantat atunci când evenimentele ajung în altă ordine sau apar cu întârziere [ABC⁺15].

2.2 Modele arhitecturale și domenii de aplicare

2.2.1 Soluții și arhitecturi existente

2.2.1.1 Apache Kafka

Apache Kafka este una dintre cele mai cunoscute platforme moderne pentru event streaming. Kafka este descris ca un sistem distribuit format din servere și clienți, care comunică printr-un protocol de rețea performant și care permite publicarea, stocarea și consumarea fluxurilor de evenimente [Apa26b].

Kafka tratează evenimentele ca înregistrări persistente într-un jurnal distribuit. Spre deosebire de unele sisteme de mesagerie unde mesajul dispare după consum, în Kafka evenimentele pot fi păstrate pentru o perioadă configurată și citite de

mai mulți consumatori. Această proprietate permite atât procesare live, cât și re-procesare a unor evenimente istorice [Apa26b].

Topicurile sunt împărțite în partiții, iar partițiile permit paralelizarea citirii și scrierii evenimentelor. Prin această organizare, un sistem poate crește capacitatea de procesare prin adăugarea mai multor consumatori sau instanțe de procesare. Partiționarea este importantă deoarece leagă scalabilitatea sistemului de modul în care evenimentele sunt distribuite [Apa26d].

Un alt concept important este offset-ul. Consumatorii urmăresc poziția până la care au citit evenimentele dintr-o partiție, iar această poziție poate fi folosită pentru recuperare după erori. În practică, offset-urile permit unui consumator să continue procesarea de la un punct cunoscut, fără să fie nevoie ca producătorul să retransmită explicit datele [Apa26b].

Din punct de vedere conceptual, Kafka se bazează pe producători, consumatori, topicuri și partiții. Producătorii trimit evenimente către topicuri, iar consumatorii citesc evenimentele din aceste topicuri. Partițiile permit împărțirea datelor pentru stocare și procesare paralelă, ceea ce contribuie la scalabilitate și toleranță la erori.

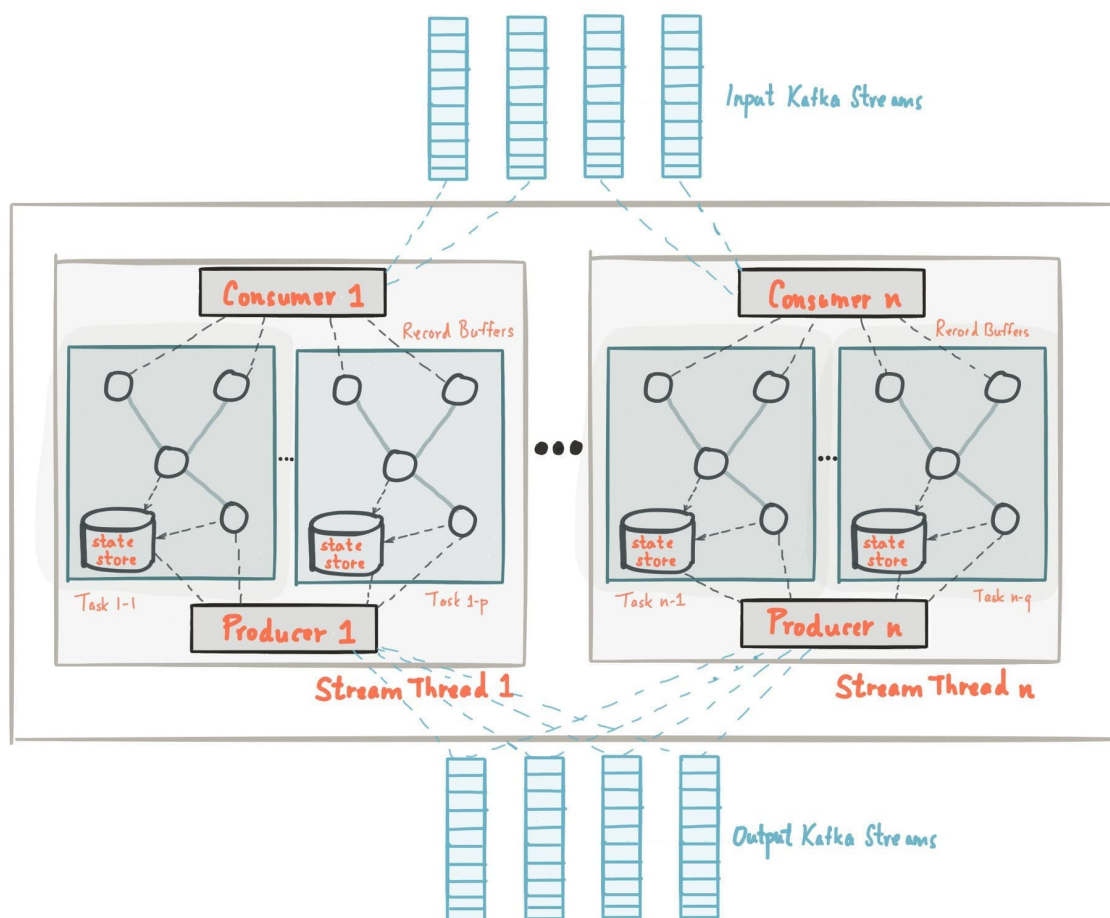


Figura 2.5: Arhitectura generală a unei aplicații Kafka Streams [Apa26d].

În Kafka Streams, partițiile sunt folosite și ca bază pentru paralelizarea procesării. Figura 2.5 prezintă o aplicație Kafka Streams care primește date din fluxuri de intrare, le procesează prin consumatori, task-uri și state stores, apoi produce rezultate către fluxuri de ieșire. Imaginea este utilă pentru a arăta că streaming-ul nu înseamnă doar transmiterea mesajelor, ci și procesarea distribuită a acestora.

Kafka este relevant pentru procesarea streaming deoarece oferă un mod standardizat de a transmite evenimente între componente independente. Prin această decuplare, producătorii nu trebuie să cunoască direct consumatorii, iar mai multe servicii pot utiliza același flux de date pentru scopuri diferite [Ama26b].

2.2.1.2 AWS Glue Jobs

AWS Glue este un serviciu complet gestionat pentru integrarea datelor și pentru procese de tip ETL, adică extract, transform și load. Într-un astfel de proces, datele sunt extrase din una sau mai multe surse, transformate într-o formă utilă și apoi încărcate într-o destinație, cum ar fi un data lake, un data warehouse sau o bază de date analitică [Ama26b].

Un AWS Glue job reprezintă logica necesară pentru realizarea procesării ETL. Documentația AWS precizează că un job poate include scripturi care conectează sursa de date, procesează datele și scriu rezultatul într-o țintă. Joburile pot fi rulate la cerere, pe baza unui program sau în urma unui eveniment [Ama26c].

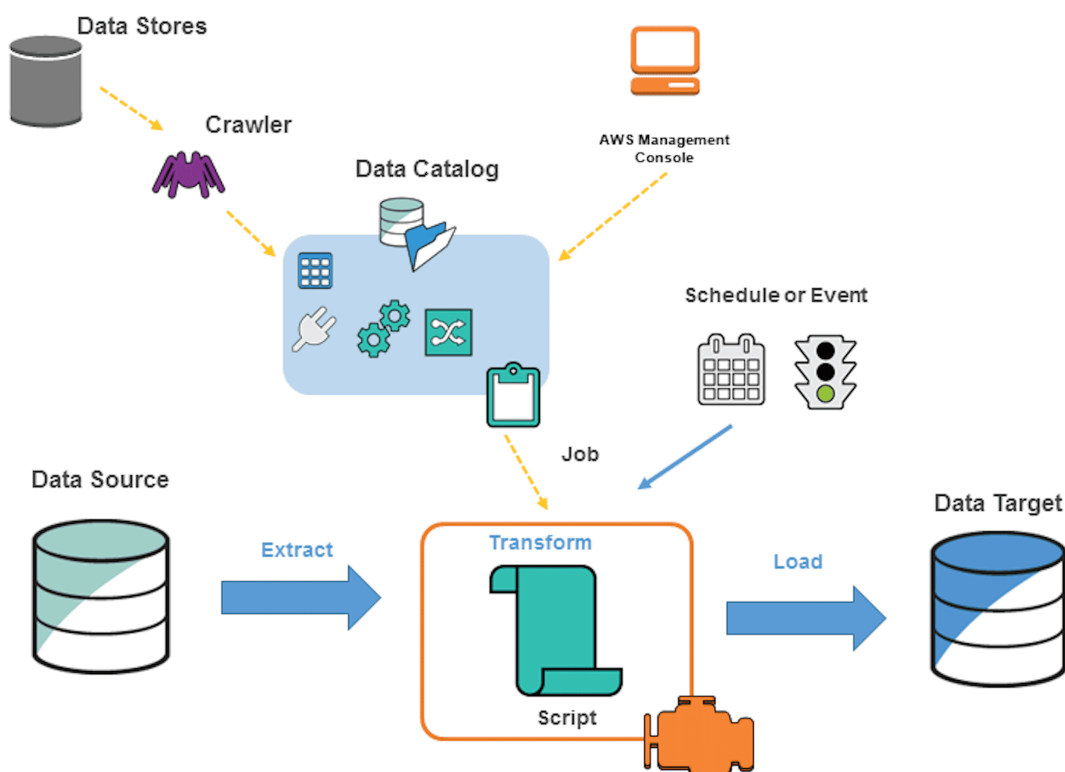


Figura 2.6: Componente principale într-un mediu AWS Glue [Ama26b].

AWS Glue include și componente precum Data Catalog, crawlers și triggers. Data Catalog păstrează metadatele despre surse și tabele, crawlers pot identifica schema datelor, iar triggers pot porni joburile pe baza unui program sau a unui eveniment. Figura 2.6 prezintă aceste componente într-un flux tipic de procesare ETL.

AWS Glue este relevant pentru procesarea batch deoarece permite definirea unor joburi care procesează date la intervale stabilite sau la cerere. Acest tip de serviciu este folosit în practică pentru pregătirea datelor, generarea de rezultate analitice și alimentarea unor sisteme de raportare, fără ca echipa să administreze direct infrastructura de procesare [Ama26a].

2.2.1.3 Arhitectura Lambda

Arhitectura Lambda a fost propusă pentru sisteme care au nevoie atât de rezultate rapide, cât și de rezultate calculate pe baza unui volum mare de date istorice. Ideea principală este împărțirea sistemului în straturi: un strat batch pentru date istorice, un strat de viteză pentru date recente și un strat de servire pentru expunerea rezultatelor [WM15].

Avantajul acestei arhitecturi este că permite folosirea punctelor forte ale ambelor paradigme. Stratul batch poate recalcula rezultate folosind întregul istoric, iar stratul de viteză poate acoperi datele recente, care nu au fost încă incluse într-o procesare batch. Figura 2.7 ilustrează această împărțire în straturi și arată modul în care datele pot ajunge la utilizator prin mai multe căi de procesare [Tex14].

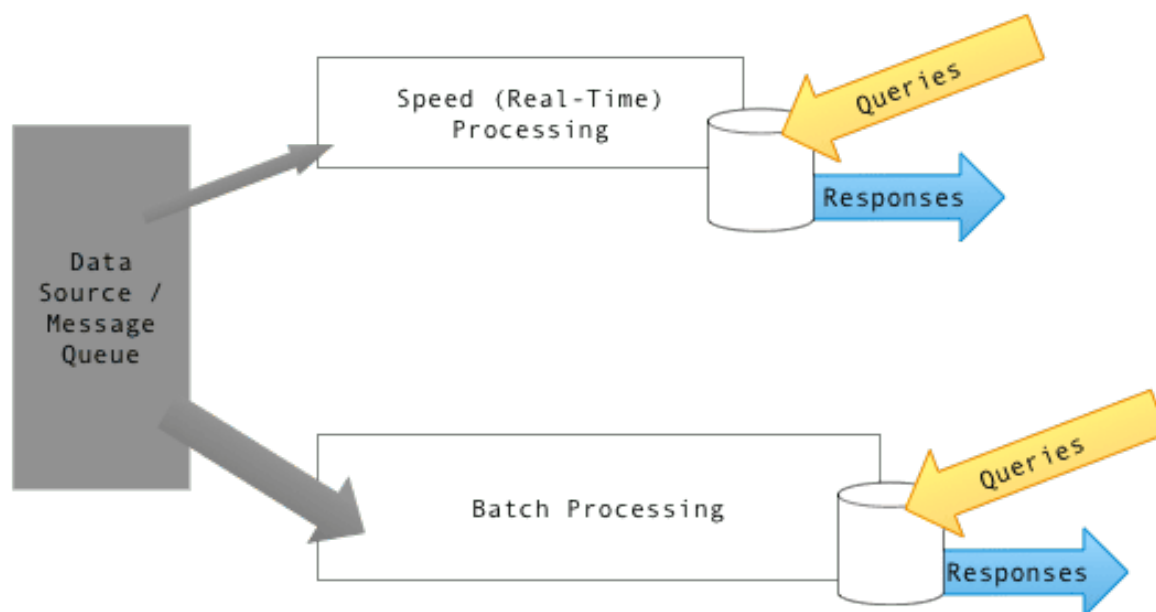


Figura 2.7: Structura generală a arhitecturii Lambda [Tex14].

Dezavantajul principal al arhitecturii Lambda este complexitatea. De multe ori, aceeași logică trebuie implementată de două ori, o dată pentru batch și o dată pentru streaming. Acest lucru poate duce la diferențe între rezultate, dificultăți de întreținere și cost operațional mai mare [Kre14].

2.2.1.4 Motoare moderne de procesare

O direcție importantă în stadiul actual al domeniului este apropierea dintre procesarea batch și procesarea streaming. Sisteme precum Apache Flink propun un model în care atât datele istorice, cât și fluxurile continue pot fi exprimate ca dataflow-uri tolerante la erori. Această abordare arată că diferența dintre batch și streaming poate fi tratată uneori la nivelul tipului de sursă de date, nu neapărat prin două modele complet separate [CKE⁺15].

Spark Streaming a propus o abordare bazată pe D-Streams, în care fluxul continuu este împărțit în secvențe mici de procesări deterministe. Această tehnică păstrează unele avantaje ale batch-ului, cum ar fi recuperarea mai simplă după erori, dar reduce latența prin procesarea datelor în intervale scurte [ZDL⁺13].

Aceste sisteme arată că în practică există mai multe puncte intermediare între batch clasic și streaming continuu. Unele aplicații au nevoie de latențe de ordinul milisecundelor, în timp ce altele pot funcționa cu latențe de câteva secunde sau minute. Alegerea tehnologiei depinde de cerințele aplicației, de volumul datelor și de nivelul de corectitudine necesar [CKE⁺15].

2.2.2 Criterii de comparație

Pentru a compara procesarea batch și procesarea streaming, este necesară stabilirea unor criterii clare. Cele mai importante criterii sunt latența, actualitatea rezultatelor, volumul de date, complexitatea, corectitudinea, toleranța la erori și tipul de scenariu pentru care fiecare abordare este potrivită [Dat26].

Latența reprezintă timpul dintre apariția datelor și disponibilitatea rezultatului. În batch, latența este de obicei mai mare, deoarece rezultatele apar după executarea unui proces periodic. În streaming, latența este mai redusă, deoarece evenimentele sunt procesate pe măsură ce ajung. Figura 2.8 evidențiază vizual această diferență, comparând procesarea datelor în loturi cu procesarea continuă a evenimentelor [Ott25].

Actualitatea rezultatelor este strâns legată de latență. Un rezultat batch poate fi corect și complet pentru perioada analizată, dar poate să nu reflecte situația curentă. Un rezultat streaming este mai apropiat de starea curentă a sistemului, dar poate fi influențat de întârzieri, ordine diferită a evenimentelor sau date incomplete [ABC⁺15].

Corectitudinea este mai ușor de controlat în batch, deoarece datele sunt disponibile înainte de procesare. În streaming, sistemul trebuie să decidă cum tratează evenimentele care ajung târziu sau în altă ordine. Din acest motiv, streaming-ul implică adesea un compromis între rapiditate și precizie [ABC⁺15].

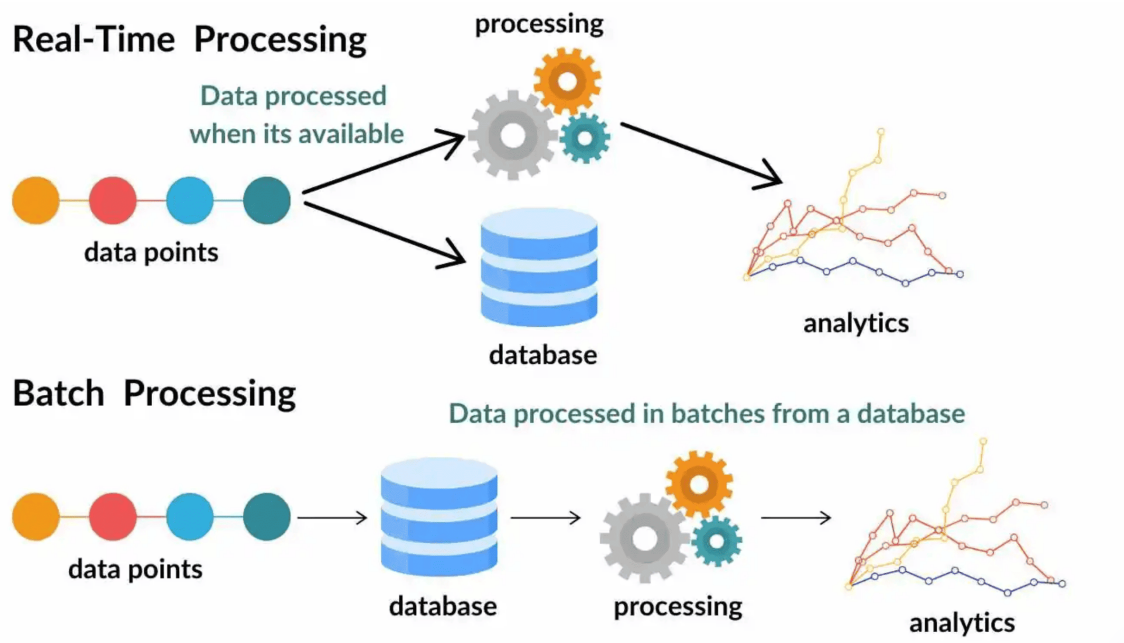


Figura 2.8: Diferența dintre procesarea batch și procesarea streaming [Ott25].

Complexitatea operațională este diferită între cele două modele. Batch-ul poate fi programat, reluat și verificat mai simplu, în timp ce streaming-ul trebuie să funcționeze continuu și să gestioneze stări, offset-uri, erori și evenimente întârziate. Prin urmare, streaming-ul oferă reacție rapidă, dar cere un nivel mai mare de proiectare și monitorizare [SÇZ05].

Costul infrastructurii depinde de modul de procesare. Batch-ul poate utiliza resurse intens doar în timpul rulării joburilor, în timp ce streaming-ul necesită componente active permanent pentru ingestie, procesare și livrare de evenimente. Această diferență este importantă în aplicații unde bugetul, scalarea și disponibilitatea sistemului influențează alegerea arhitecturii [K⁺19].

2.2.3 Exemple de utilizare

Procesarea batch este folosită în special atunci când rezultatele nu trebuie să fie disponibile imediat. Exemplele includ rapoarte financiare, statistici zilnice, prelucrarea logurilor, recalcularea unor indicatori de business, analiza datelor istorice și generarea unor rapoarte periodice [Ama26e].

Procesarea streaming este folosită în scenarii în care reacția rapidă este importantă. Exemplele includ detectarea fraudelor, monitorizarea infrastructurii, analiza activității utilizatorilor, urmărirea flotelor de vehicule, monitorizarea senzorilor industriali și sistemele care afișează date live [Apa26b].

În multe aplicații reale, cele două abordări sunt folosite împreună: streaming-ul oferă reacții rapide la evenimente importante, iar batch-ul oferă analize și statistici pe baza datelor acumulate [WM15].

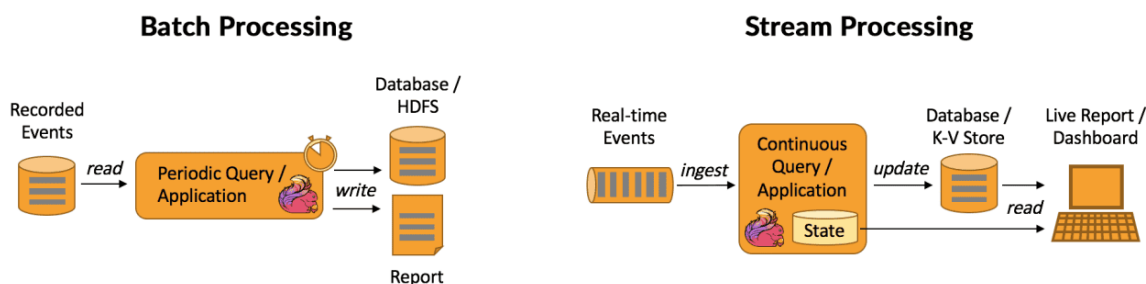


Figura 2.9: Comparație între procesarea batch și procesarea streaming [Dee24].

Sistemele de monitorizare sunt potrivite pentru explicarea diferenței dintre batch și streaming deoarece combină două nevoi diferite. Pe de o parte, ele trebuie să observe rapid schimbările importante. Pe de altă parte, trebuie să permită analiza datelor acumulate pentru identificarea unor tendințe, generarea de rapoarte sau evaluarea evoluției în timp [PMdS⁺22].

În zona medicală, sistemele de monitorizare pot colecta parametri fiziologici precum pulsul, temperatura, tensiunea arterială sau nivelul de oxigen. Studiile existente descriu astfel de sisteme ca fiind orientate spre colectarea datelor, analizarea lor și generarea de avertizări atunci când apar situații relevante [PMdS⁺22].

Totuși, nu toate informațiile dintr-un sistem de monitorizare trebuie procesate în timp real. Pentru analiza evoluției unui pacient, încărcarea unui departament sau calcularea unor valori medii, procesarea batch poate fi suficientă. În aceste cazuri, accentul nu este pus pe reacția imediată, ci pe obținerea unei imagini mai complete asupra datelor acumulate [Dat26].

Astfel, sistemele de monitorizare arată clar complementaritatea dintre cele două paradigme. Streaming-ul răspunde la întrebarea „ce se întâmplă acum?“, în timp ce batch-ul răspunde la întrebarea „ce s-a întâmplat într-o perioadă?“. Această diferență este importantă pentru înțelegerea modului în care cele două modele pot fi alese sau combinate în funcție de cerințele unei aplicații [WM15].

În sistemele de monitorizare medicală, diferența dintre batch și streaming poate fi observată natural. Măsurătorile fiziologice pot produce fluxuri continue de date,

iar anumite valori trebuie analizate rapid pentru a genera avertizări. În același timp, evoluția pacientului, sumarul unei internări sau analiza tratamentelor necesită o perspectivă retrospectivă asupra datelor acumulate [PMdS+22].

Această complementaritate explică de ce un scenariu medical este potrivit pentru demonstrarea celor două paradigme. Streaming-ul este asociat cu alerte și monitorizare curentă, iar batch-ul este asociat cu rapoarte, agregări și interpretări istorice. Cele două abordări nu se exclud, ci pot fi combinate într-un sistem care are atât componentă operațională, cât și componentă analitică [WM15].

2.2.4 Moduri actuale de explicare

În materialele tehnice și educaționale, diferența dintre batch și streaming este explicată frecvent prin momentul procesării și prin tipul datelor analizate. Batch processing este prezentat ca procesare periodică a unor seturi finite de date, în timp ce stream processing este prezentat ca procesare continuă a evenimentelor care apar fără un final prestabilit [Dat26, Con26].

O analogie simplă este diferența dintre raport și alertă. Un raport este util pentru analiză și poate fi generat după acumularea datelor, în timp ce o alertă are valoare doar dacă este produsă rapid. Această diferență este importantă în procesarea în timp real, unde latența redusă este una dintre cerințele principale ale sistemelor de stream processing [SCZ05].

Explicațiile existente devin adesea tehnice atunci când introduc noțiuni precum clustere, brokeri, partiții, offset-uri, checkpointing, replicare sau garanții de livrare. Aceste concepte sunt necesare pentru implementare, dar pot fi dificile pentru o persoană care întâlnește pentru prima dată diferența dintre batch și streaming [Apa26d].

De aceea, în această lucrare am folosit un scenariu vizual de monitorizare, în care diferența dintre alertă și raport poate fi observată mai direct. Streaming-ul este asociat cu evenimente recente și alerte, iar batch-ul este asociat cu statistici, rapoarte și interpretarea datelor acumulate [PMdS+22].

Din aspectele prezentate rezultă că procesarea batch și procesarea streaming nu trebuie privite ca tehnologii concurente, ci ca abordări complementare. Streaming-ul susține reacția rapidă la evenimente recente, în timp ce batch-ul oferă rezultate agregate și o perspectivă stabilă asupra datelor acumulate.

Această delimitare oferă baza pentru alegerea tehnologiilor folosite în MedStream, unde cele două paradigme sunt integrate într-un sistem demonstrativ construit în jurul acelorași date simulate.

Capitolul 3

Tehnologii utilizate pentru aplicație

Dezvoltarea aplicației MedStream a presupus alegerea unor tehnologii potrivite pentru o aplicație full-stack și pentru integrarea procesării streaming cu procesarea batch. Sistemul include o interfață web pentru utilizator, un backend pentru logica aplicației, o bază de date pentru persistență, o infrastructură de mesagerie pentru evenimente și componente pentru procesarea periodică a datelor.

Tehnologiile prezentate în continuare nu sunt tratate izolat, ci prin rolul lor în cadrul aplicației. FastAPI, React, PostgreSQL, Kafka, WebSocket, APScheduler și Docker contribuie la separarea responsabilităților dintre interfață, backend, persistență, comunicare live și procesare batch.

3.1 Tehnologii pentru backend

3.1.1 Python

Backend-ul aplicației MedStream este dezvoltat în Python, deoarece aplicația are nevoie atât de expunerea unor API-uri, cât și de componente care lucrează cu date, evenimente și procese periodice. Python oferă un ecosistem potrivit pentru acest tip de sistem, prin biblioteci pentru aplicații web, validare, acces la baze de date, procesare de date și automatizarea unor sarcini.

În MedStream, Python are rolul de limbaj principal pentru logica backend-ului. Acesta este folosit pentru definirea rutelor API, validarea datelor primite, comunicarea cu baza de date, integrarea cu Kafka, pornirea proceselor batch și gestionarea simulatorului clinic. Alegerea unui singur limbaj pentru aceste componente simplifică dezvoltarea și reduce diferența dintre partea operațională a aplicației și partea de procesare a datelor.

3.1.2 Virtual environments

Pentru organizarea dependențelor backend-ului, proiectul folosește un mediu virtual Python. Modulul `venv` permite izolarea bibliotecilor necesare aplicației față de cele instalate la nivelul sistemului de operare [Py26].

În cazul MedStream, această izolare este utilă deoarece backend-ul folosește mai multe biblioteci externe, precum FastAPI, SQLAlchemy, Pydantic, componente pentru Kafka și mecanisme de programare a joburilor. Mediul virtual face configurarea proiectului mai clară și reduce riscul conflictelor între versiuni diferite ale acelorași pachete.

3.1.3 FastAPI și REST API

FastAPI a fost ales pentru backend deoarece permite construirea rapidă a unor API-uri clare, cu validare integrată și documentație generată automat. Framework-ul folosește type hints standard din Python și este potrivit pentru aplicații în care datele primite de la client trebuie verificate înainte de a fi procesate [Fas26b].

În MedStream, FastAPI reprezintă stratul prin care frontend-ul comunică cu logica aplicației. Prin rutele definite în backend, interfața poate crea și citi pacienți, poate obține valori vitale, alerte, metrice și rezultate batch. Această separare permite ca frontend-ul să rămână responsabil doar de afișare și interacțiune, iar regulile aplicației să fie păstrate în backend.

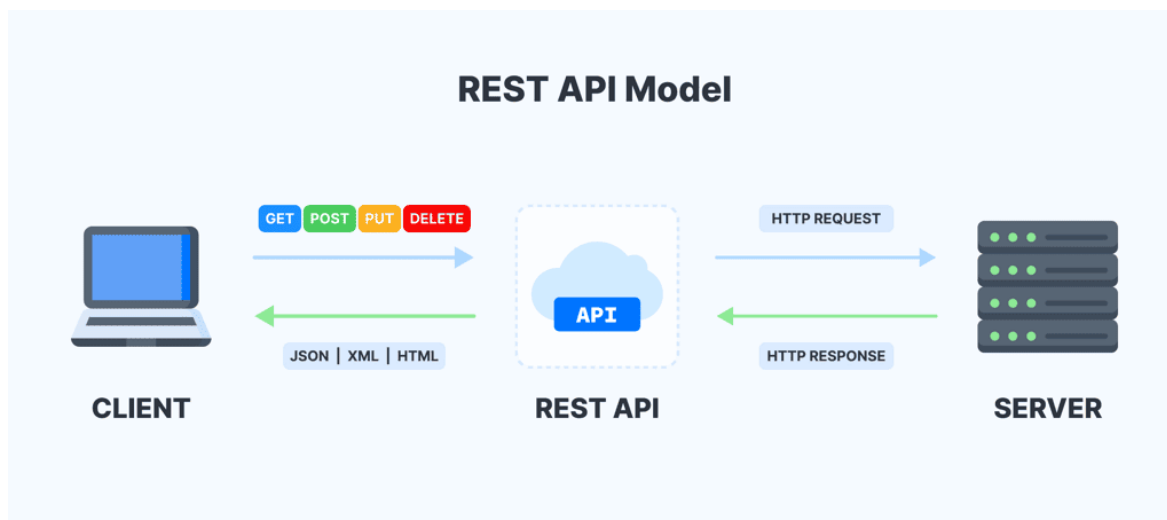


Figura 3.1: Model general de comunicare prin REST API [iDe25].

Modelul REST API este folosit pentru operațiile obișnuite ale aplicației, unde frontend-ul trimite o cerere, iar backend-ul returnează un răspuns. Figura 3.1 pre-

zintă un model simplificat în care o aplicație client comunică prin HTTP cu un API, iar acesta interacționează cu logica și datele sistemului [iDe25].

3.1.4 Pydantic

Pydantic este folosit în MedStream pentru definirea schemelor de date și pentru validarea informațiilor care intră sau ies din backend. În aplicațiile construite cu FastAPI, Pydantic este integrat în mod natural pentru descrierea request-urilor și response-urilor [Fas26b].

Rolul său este important deoarece aplicația lucrează cu entități structurate, precum doctori, pacienți, valori vitale, alerte și rezultate analitice. Prin folosirea unor modele clare, backend-ul poate verifica tipurile câmpurilor, valorile obligatorii și forma obiectelor înainte ca acestea să fie salvate în baza de date sau trimise către frontend.

3.1.5 CRUD operations

Operațiile CRUD sunt folosite în MedStream pentru administrarea datelor principale ale aplicației. Acestea permit crearea, citirea, actualizarea și ștergerea resurselor, cum ar fi pacienții, informațiile despre doctori sau alte date operaționale necesare scenariului de spital simulat.

În arhitectura aplicației, frontend-ul nu accesează direct baza de date. Acesta trimite cereri către backend, iar backend-ul validează datele, aplică regulile aplicației și execută operațiile necesare prin modelele și serviciile definite. Această separare ajută la menținerea unei structuri clare și la centralizarea logicii aplicației.

3.1.6 CORS middleware

Middleware-ul CORS este necesar deoarece frontend-ul și backend-ul pot rula pe origini diferite, mai ales în dezvoltarea locală, unde aplicația React și serverul FastAPI folosesc porturi diferite. Documentația FastAPI explică faptul că această situație apare frecvent atunci când componentele web rulează pe domenii, protocoale sau porturi diferite [Fas26a].

În MedStream, configurarea CORS permite browserului să accepte cererile trimise de frontend către backend. Astfel, interfața poate comunica în mod controlat cu API-ul aplicației, fără ca backend-ul să accepte cereri din orice sursă.

3.2 Tehnologii pentru frontend

3.2.1 JavaScript

JavaScript este una dintre tehnologiile principale ale aplicațiilor web moderne, fiind utilizat pentru dezvoltarea interfețelor interactive care rulează în browser. Într-o aplicație full-stack, JavaScript permite construirea unei interfețe dinamice, capabile să comunice cu serverul, să actualizeze datele afișate și să reacționeze la acțiunile utilizatorului.

Rolul JavaScript în arhitectura sistemului este legat de partea de prezentare și interacțiune. Prin intermediul său, frontend-ul poate solicita date de la API, poate procesa răspunsurile primite și poate actualiza componentele vizuale fără reîncărcarea completă a paginii.

3.2.2 React

React este o bibliotecă pentru construirea interfețelor web pe baza unor componente reutilizabile. Documentația oficială descrie React ca o bibliotecă pentru interfețe construite din piese individuale, numite componente, care pot fi combinate pentru a forma pagini și aplicații complete [Rea26c].

Utilizarea React este potrivită într-o aplicație full-stack deoarece interfața poate fi împărțită în componente distincte, fiecare având un rol clar. Această organizare contribuie la separarea responsabilităților, la reutilizarea codului și la actualizarea eficientă a interfeței atunci când datele se modifică.

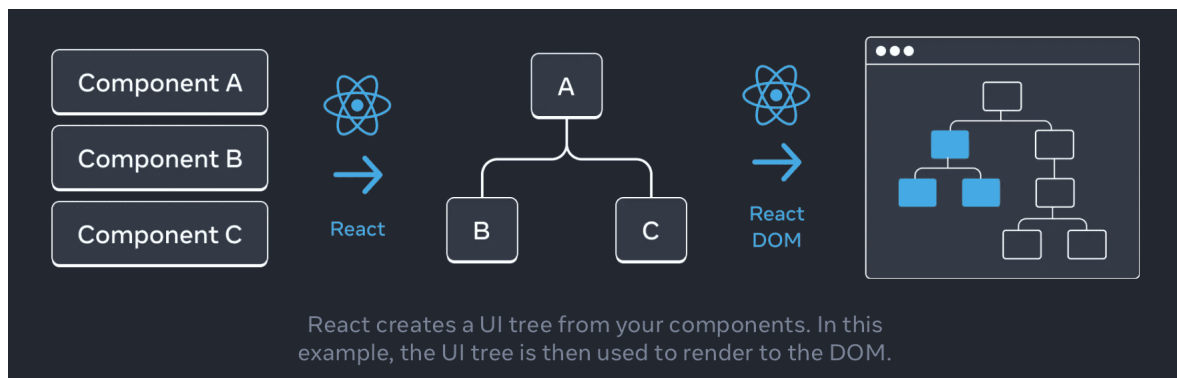


Figura 3.2: Organizare a interfeței ca arbore de componente în React [Rea26d].

Figura 3.2 ilustrează modul în care o interfață React poate fi privită ca un arbore de componente. Această perspectivă este utilă pentru aplicații cu mai multe pagini și zone vizuale, deoarece fiecare parte a interfeței poate fi construită, testată și reutilizată separat.

3.2.3 Single Page Application

O Single Page Application este o aplicație web care încarcă o singură pagină HTML și actualizează conținutul acesteia dinamic, folosind JavaScript, fără reîncărcarea completă a paginii la fiecare schimbare de ecran. MDN descrie acest model ca o aplicație care încarcă un singur document și actualizează conținutul paginii atunci când trebuie afișate informații noi [Moz26b].

Acest model este potrivit pentru aplicații interactive, deoarece navigarea poate fi realizată rapid, iar componentele pot fi actualizate pe baza datelor primite de la backend. Într-o aplicație de monitorizare, o interfață de tip SPA permite afișarea fluidă a dashboard-ului, listelor, paginilor de detalii și metricilor.

3.2.4 JSX

JSX este o extensie de sintaxă utilizată frecvent împreună cu React pentru descrierea structurii interfeței. Prin JSX, elementele vizuale pot fi scrise într-o formă apropiată de HTML, dar integrate direct în codul JavaScript al componentelor. Documentația React prezintă JSX ca mod de a combina markup-ul cu logica de randare a componentelor [Rea26b].

Utilizarea JSX este utilă deoarece face componentele mai ușor de citit și de organizat. În loc ca structura interfeței și logica acesteia să fie complet separate, componenta poate conține atât datele necesare, cât și modul în care acestea sunt afișate.

3.2.5 React Hooks

React Hooks sunt funcții care permit utilizarea unor caracteristici React direct în componente funcționale. Documentația precizează că hook-urile permit folosirea stării, efectelor și a altor mecanisme React în interiorul componentelor [Rea26a].

Într-o aplicație interactivă, hook-uri precum `useState` și `useEffect` sunt utile pentru gestionarea stării locale, încărcarea datelor și reacția la schimbări. De asemenea, hook-urile personalizate pot grupa logică reutilizabilă, cum ar fi obținerea datelor de la API sau gestionarea conexiunilor live.

3.2.6 Design system și stilizare

Interfața aplicației MedStream folosește Cloudscape Design Components împreună cu fișiere CSS personalizate. Această abordare permite utilizarea unor componente vizuale consistente, potrivite pentru aplicații de tip dashboard, administrare și monitorizare, dar păstrează în același timp libertatea de a adapta designul la nevoile specifice aplicației.

Cloudscape Design Components. Cloudscape este folosit ca bază pentru componente de interfață inspirate din ecosistemul AWS. Acesta oferă componente potrivite pentru aplicații complexe, precum layout-uri, tabele, formulare, butoane, carduri, elemente de navigare și zone de conținut structurat [Clo26]. În MedStream, această alegere ajută la construirea unei interfețe familiare pentru utilizatori obișnuiți cu aplicații cloud sau panouri administrative.

CSS personalizat. Pe lângă componentele Cloudscape, aplicația folosește fișiere CSS organizate în funcție de pagini și zone funcționale. Această organizare permite păstrarea stilurilor specifice aproape de paginile pe care le influențează și face mai ușoară întreținerea interfeței. CSS-ul personalizat este folosit pentru ajustări de layout, culori de severitate, stilizarea graficelor, spațieri, comportamente vizuale și elemente specifice scenariului de monitorizare clinică simulată.

Prin combinarea Cloudscape cu CSS personalizat, aplicația păstrează o bază vizuală coerentă, dar nu devine dependentă exclusiv de un singur set de componente. Cloudscape oferă structura și familiaritatea unei interfețe de tip enterprise, iar CSS-ul personalizat permite adaptarea designului la identitatea și fluxurile specifice MedStream.

3.2.7 Recharts

Recharts este o bibliotecă de vizualizare a datelor construită pentru React și orientată spre utilizarea componentelor reutilizabile. Aceasta permite realizarea unor grafice precum diagrame liniare, coloane, arii sau diagrame circulare, folosind o sintaxă apropiată de restul componentelor din interfață [Rec26].

Alegerea acestei biblioteci este potrivită într-o aplicație care pune accent pe afișarea datelor, deoarece facilitează integrarea vizualizărilor direct în structura frontend-ului. Graficele pot primi date din backend și pot fi actualizate dinamic, ceea ce susține o interfață clară și ușor de urmărit.

3.3 Tehnologii pentru comunicarea dintre componente

3.3.1 HTTP și API-uri REST

În MedStream, comunicarea principală dintre frontend și backend se realizează prin cereri HTTP către API-uri REST. Acest model este potrivit pentru acțiunile obișnuite ale aplicației, cum ar fi autentificarea, încărcarea listelor de pacienți, salvarea modificărilor sau afișarea metricilor existente. HTTP funcționează pe baza unui model request-response, în care clientul trimite o cerere, iar serverul returnează un răspuns [Moz26a].

Pentru aplicația MedStream, REST este util deoarece oferă o separare clară între interfață și logica backend. Frontend-ul nu are nevoie să cunoască modul în care sunt organizate tabelele sau serviciile interne, ci consumă doar endpoint-urile expuse de backend. Acest lucru face ca aplicația să fie mai ușor de întreținut și de extins.

3.3.2 WebSocket

Pentru actualizările care trebuie afișate rapid, MedStream folosește WebSocket. Spre deosebire de cererile HTTP obișnuite, unde frontend-ul trebuie să ceară explicit date noi, WebSocket permite menținerea unei conexiuni active între browser și server. Prin această conexiune, serverul poate trimite mesaje către client atunci când apar evenimente noi [Moz26c].

În contextul aplicației, WebSocket este folosit pentru actualizări live, precum valori vitale noi, alerte sau schimbări de stare. Această alegere este importantă pentru partea de streaming a sistemului, deoarece interfața poate reacționa rapid la evenimente fără reîncărcarea paginii și fără interogări repetate către backend.

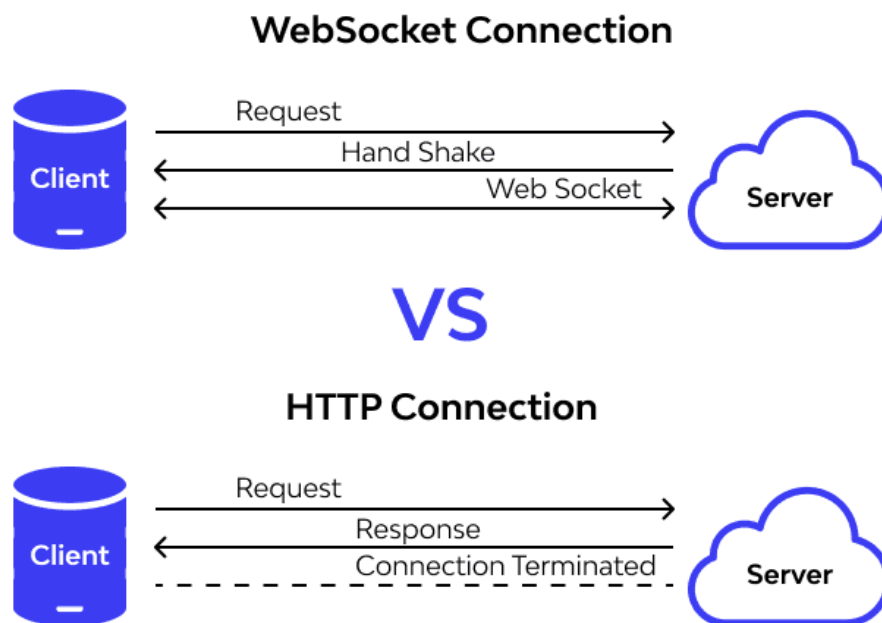


Figura 3.3: Comunicarea între client și server prin WebSocket [Moz26c].

3.3.3 Securitate, validare și integritatea datelor

Într-o aplicație care gestionează date despre pacienți, chiar și într-un scenariu simulat, validarea și integritatea datelor sunt importante. Datele trimise din frontend trebuie verificate înainte de procesare, pentru a evita valori incomplete, invalide sau incompatibile cu regulile aplicației. FastAPI și Pydantic permit definirea unor modele de date și validarea automată a câmpurilor primite [Fas26b].

La nivel de comunicare, un sistem real ar trebui să folosească HTTPS pentru cererile HTTP și WSS pentru conexiunile WebSocket. Aceste mecanisme protejează datele transmise între client și server și sunt importante în aplicațiile care lucrează cu informații personale sau sensibile [Moz26c].

Integritatea datelor este susținută și prin baza de date relațională. Entități precum doctorii, pacienții, valorile vitale și alertele trebuie identificate clar și legate corect între ele. Constrângeri precum cheile primare, cheile străine și regulile de unicitate ajută la păstrarea unei structuri coerente a datelor [Pos26].

3.4 Tehnologii pentru persistența datelor

3.4.1 PostgreSQL

PostgreSQL a fost ales ca bază de date pentru MedStream deoarece aplicația lucrează cu date structurate și cu relații clare între entități. Doctorii, pacienții, valorile vitale, alertele, diagnosticele, medicațiile și istoricul internărilor pot fi organizate natural în tabele relaționale. Documentația PostgreSQL descrie sistemul ca fiind bazat pe modelul client-server, în care aplicațiile client comunică cu serverul de baze de date pentru executarea interogărilor și tranzacțiilor [Pos26].

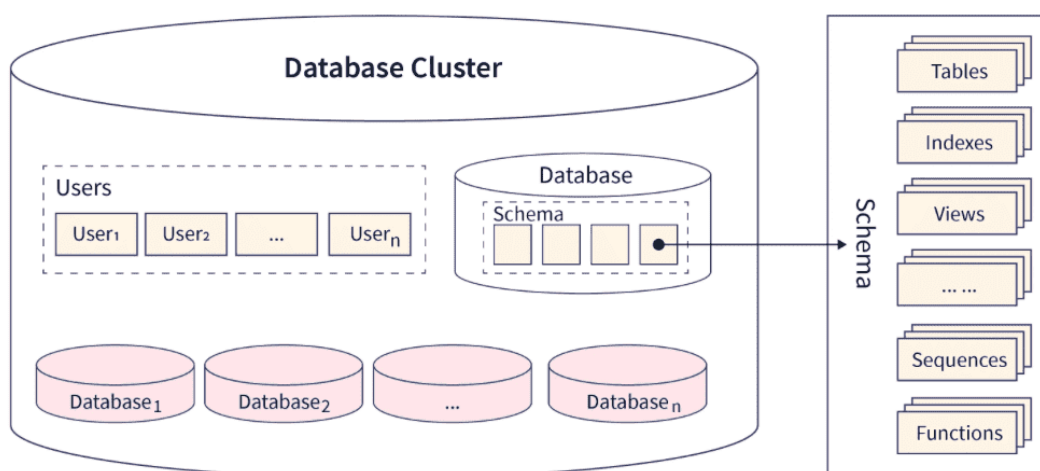


Figura 3.4: Structura fizică a datelor în PostgreSQL [Sca24].

În MedStream, PostgreSQL susține atât partea operațională, cât și partea analitică a aplicației. Datele despre pacienți, valori vitale și alerte sunt folosite pentru funcționarea curentă a sistemului, iar rezultatele batch și snapshot-urile analitice sunt păstrate pentru afișarea metricilor și a analizelor periodice. Această alegere permite ca aceeași bază de date să fie folosită atât pentru stocarea datelor curente, cât și pentru rezultatele agregate.

Un alt motiv pentru folosirea PostgreSQL este suportul pentru tranzacții și constrângeri de integritate. Aceste mecanisme sunt utile într-un sistem în care relațiile dintre pacienți, doctori, alerte și date clinice trebuie păstrate coerent. Figura 3.4 ilustrează structura fizică a datelor în PostgreSQL, incluzând concepte precum cluster, database, schema, table, index și row [Sca24].

3.4.2 SQLAlchemy ORM

SQLAlchemy ORM este folosit pentru legătura dintre codul Python și baza de date relațională. Prin ORM, entitățile aplicației pot fi reprezentate ca clase Python, iar tabelele din baza de date pot fi gestionate prin modele și relații definite în cod. Documentația SQLAlchemy descrie stilul declarativ ca principalul mod de definire a claselor mapate [SQL26].

În MedStream, această abordare simplifică lucrul cu entități precum doctorul, pacientul, alerta sau valoarea vitală. Backend-ul poate lucra cu obiecte Python, iar SQLAlchemy se ocupă de transformarea operațiilor în interogări către baza de date. Astfel, codul aplicației rămâne mai ușor de urmărit decât în cazul în care toate operațiile ar fi scrise direct în SQL.

3.5 Procesarea evenimentelor și a datelor

3.5.1 Apache Kafka

Apache Kafka este folosit în MedStream ca infrastructură pentru transportul evenimentelor generate de sistem. În loc ca simulatorul, backend-ul și componentele care procesează datele să depindă direct unele de altele, evenimentele sunt publicate în topicuri Kafka și consumate de componentele interesate. Kafka permite publicarea, stocarea și procesarea fluxurilor de evenimente atât pe măsură ce apar, cât și retrospectiv [Apa26b].

Alegerea Kafka este potrivită pentru partea de streaming a aplicației deoarece decuplează producerea valorilor vitale de procesarea lor. Simulatorul poate genera evenimente, iar consumatorul Kafka le poate prelua pentru detectarea alertelor și actualizarea sistemului. Această separare face arhitectura mai clară și evidențiază

modul în care datele pot circula printr-un sistem bazat pe evenimente.

În MedStream, Kafka are rolul de strat intermediar între sursele de date simulate și componentele care reacționează la aceste date. Evenimentele recente pot fi procesate rapid pentru alerte și actualizări live, iar datele persistente pot fi folosite ulterior pentru calcule batch. Astfel, Kafka susține partea de demonstrație a procesării streaming, fără a înlocui baza de date sau mecanismul batch.

3.5.2 APScheduler

APScheduler este folosit pentru pornirea periodică a proceselor batch din aplicație. În loc ca aceste calcule să fie executate manual, scheduler-ul permite rularea lor la intervale stabilite, ceea ce se potrivește cu ideea de procesare periodică a datelor acumulate [APS26].

În MedStream, APScheduler are rolul de a declanșa calcule precum agregări, metrici și rezultate analitice. Această alegere permite separarea fluxului live de fluxul batch: evenimentele recente sunt tratate prin streaming, iar datele deja salvate sunt analizate periodic prin joburi programate.

3.5.3 Apache Spark

În această lucrare, Apache Spark este prezentat ca tehnologie relevantă pentru scalarea procesării batch, însă implementarea demonstrativă MedStream folosește un mecanism batch local, coordonat prin APScheduler. Din acest motiv, Spark nu este tratat ca o componentă executată direct în prototip, ci ca reper arhitectural pentru extinderea procesării asupra unor volume mai mari de date.

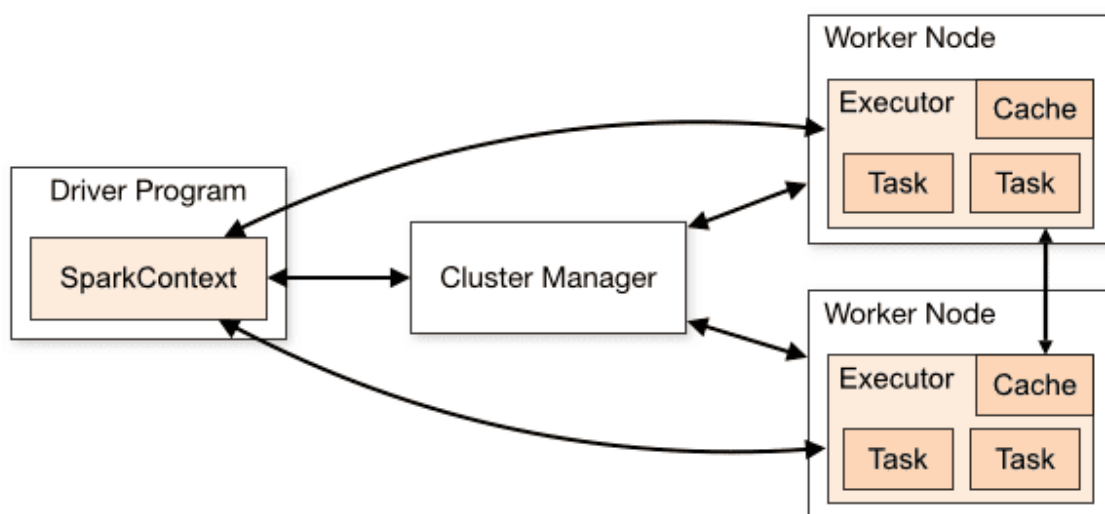


Figura 3.5: Arhitectură Spark în cluster [Apa26c].

Această delimitare este importantă deoarece MedStream urmărește să demonstreze diferența dintre procesarea imediată a evenimentelor și procesarea periodică a datelor acumulate, fără a reproduce infrastructura completă a unui cluster Spark. În contextul lucrării, arhitectura Spark ajută la înțelegerea modului în care procesarea batch ar putea fi extinsă ulterior, dacă aplicația ar lucra cu volume mai mari de date [Apa26e].

În arhitectura MedStream, Spark este relevant pentru componenta analitică și pentru ideea de procesare batch. Chiar dacă aplicația are scop demonstrativ, includerea unei componente orientate spre procesarea datelor ajută la evidențierea diferenței dintre evenimentele tratate imediat și calculele realizate periodic pe date acumulate.

Figura 3.5 prezintă modelul general de execuție Spark într-un cluster. Driverul coordonează aplicația, cluster managerul alocă resurse, iar nodurile de lucru rulează executori care execută task-uri și pot păstra date în cache. În contextul lucrării, această arhitectură este importantă pentru înțelegerea modului în care procesarea batch poate fi extinsă către volume mai mari de date.

3.6 Containerizare și rulare locală

Docker este folosit pentru rularea locală a componentelor MedStream într-un mod izolat și reproductibil. Aplicația depinde de mai multe servicii, precum backend, frontend, PostgreSQL, Kafka și alte componente auxiliare, iar containerizarea permite pornirea lor într-un mod mai controlat. Documentația Docker explică faptul că platforma permite separarea aplicațiilor de infrastructură și rularea lor în medii izolate [Doc26].

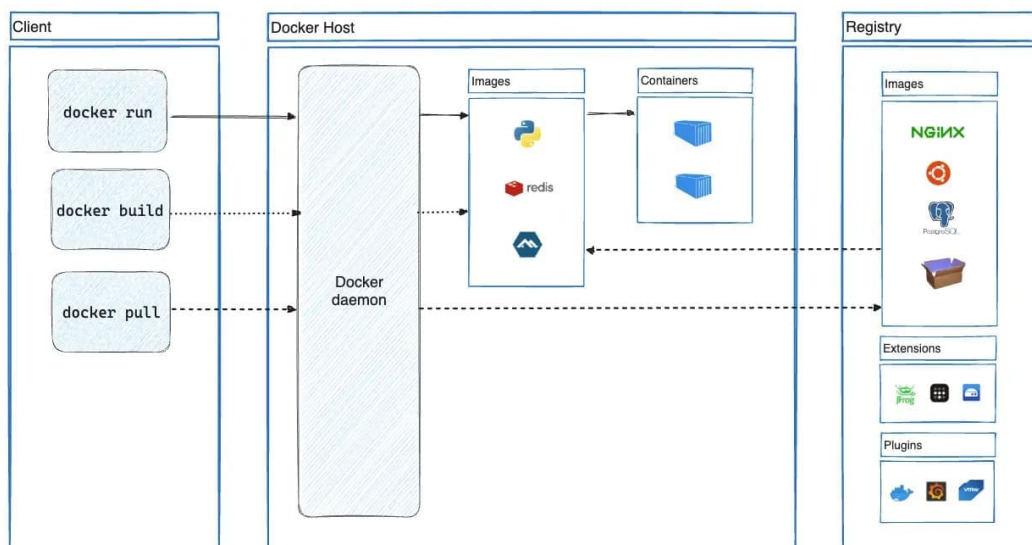


Figura 3.6: Arhitectura generală Docker [Doc26].

În cazul MedStream, Docker nu este folosit doar ca tehnologie auxiliară, ci ca mecanism care simplifică testarea arhitecturii complete. Fiecare serviciu poate rula într-un container propriu, cu dependențele necesare, ceea ce reduce pașii de configurare manuală și face mai ușoară reproducerea mediului de dezvoltare.

Docker folosește o arhitectură client-server, în care clientul comunică cu Docker daemon pentru a construi imagini, a porni containere și a gestiona obiectele Docker. Figura 3.6 prezintă relația dintre client, daemon, imagini, containere și registru. Pentru această lucrare, partea importantă este faptul că această arhitectură permite rularea coordonată a mai multor servicii necesare demonstrației.

3.7 Integrarea tehnologiilor

Tehnologiile folosite în MedStream sunt organizate pe straturi, fiecare având un rol clar în aplicație. Frontend-ul gestionează interacțiunea cu utilizatorul, backend-ul coordonează logica aplicației, baza de date păstrează informațiile persistente, Kafka susține fluxul de evenimente, iar mecanismele batch calculează rezultate periodice. Această organizare este apropiată de modelul three-tier architecture, în care aplicația este împărțită în strat de prezentare, strat de aplicație și strat de date [IBM26].

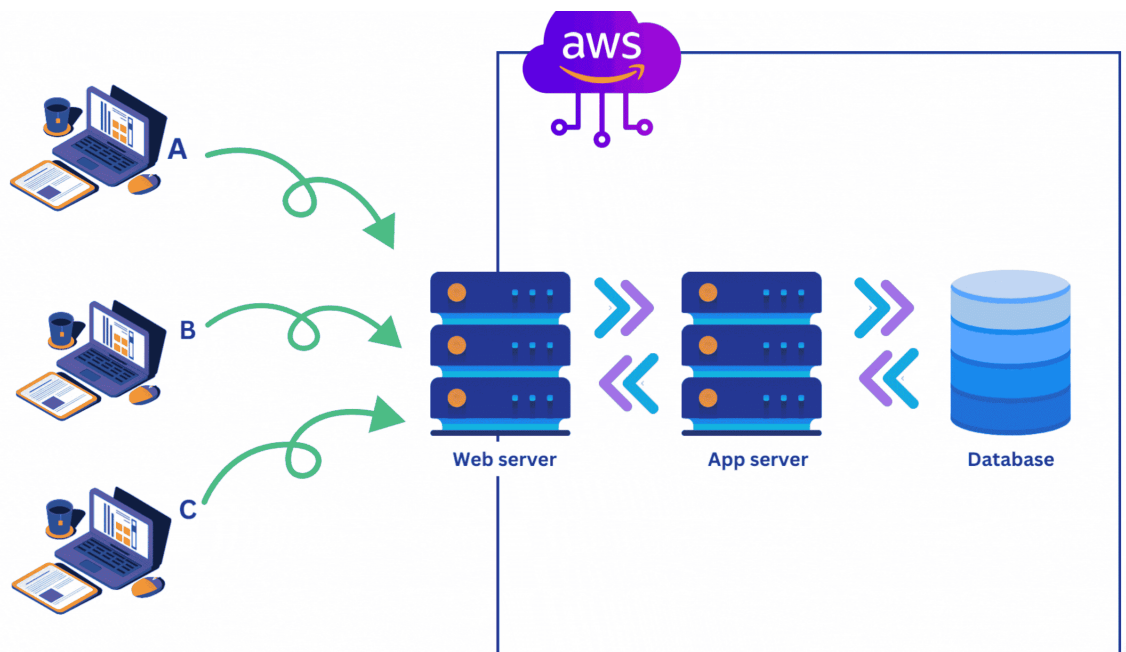


Figura 3.7: Model de arhitectură pe trei straturi [Kum23].

Separarea acestor straturi este importantă deoarece MedStream urmărește să demonstreze folosirea împreună a procesării streaming și batch. Streaming-ul este in-

tegrat pentru evenimente recente și actualizări live, în timp ce batch-ul este integrat pentru agregări și analize asupra datelor acumulate. Astfel, tehnologiile nu sunt folosite doar pentru că sunt moderne, ci pentru că susțin roluri diferite în arhitectura aplicației.

Figura 3.7 prezintă modelul general al unei arhitecturi pe trei straturi. În MedStream, acest model este extins prin includerea componentelor de mesagerie și procesare, necesare pentru comparația dintre fluxurile live și calculele periodice [Kum23].

Tehnologiile discutate formează baza tehnică pe care este construită aplicația MedStream. Separarea dintre frontend, backend, bază de date, mesagerie și mecanisme de procesare permite integrarea fluxului live cu procesarea periodică fără amestecarea responsabilităților într-un singur modul.

Pe această bază tehnologică poate fi descrisă implementarea propriu-zisă a aplicației, de la organizarea componentelor până la fluxurile prin care datele simulate sunt transformate în alerte, metrice și rezultate agregate.

Capitolul 4

Implementarea aplicației MedStream

Implementarea MedStream urmărește integrarea componentelor prezentate anterior într-o aplicație funcțională, capabilă să genereze date clinice sintetice, să proceseze evenimente recente și să calculeze rezultate periodice pe baza datelor acumulate. Accentul cade pe modul în care componentele backend, frontend, baza de date, Kafka, WebSocket, simulatorul clinic și mecanismul batch colaborează în același sistem demonstrativ.

Aplicația este construită ca un sistem hibrid, în care procesarea streaming și procesarea batch sunt folosite împreună. Streaming-ul susține monitorizarea live și generarea alertelor, iar batch-ul susține metricile, agregările și analiza retrospectivă. Această organizare permite observarea diferenței dintre reacția imediată la evenimente și interpretarea periodică a datelor persistate.

4.1 Prezentare generală și arhitectură

MedStream este o aplicație full-stack pentru management operațional într-un scenariu de spital simulat. Aplicația combină administrarea pacienților, monitorizarea valorilor vitale, generarea alertelor, analiza tratamentelor și afișarea metricilor streaming și batch. Frontend-ul este dezvoltat în React, Vite, React Router și Cloudscape, folosind fișiere CSS personalizate. Backend-ul este dezvoltat în Python, cu FastAPI, SQLAlchemy și Pydantic.

Arhitectura aplicației este împărțită în frontend, backend, bază de date, infrastructură de streaming și mecanism batch. Frontend-ul trimite cereri REST către backend și ascultă un canal WebSocket pentru actualizări live. Backend-ul validează cererile, salvează datele în PostgreSQL, consumă evenimente Kafka și transmite actualizări către interfață. Kafka este folosit pentru transportul evenimentelor de valori vitale și alerte, iar WebSocket transformă aceste evenimente în modificări vizibile rapid în browser. La pornire, backend-ul inițializează baza de date, verifică

topicurile Kafka, pornește consumer-ul, simulatorul clinic și scheduler-ul batch.

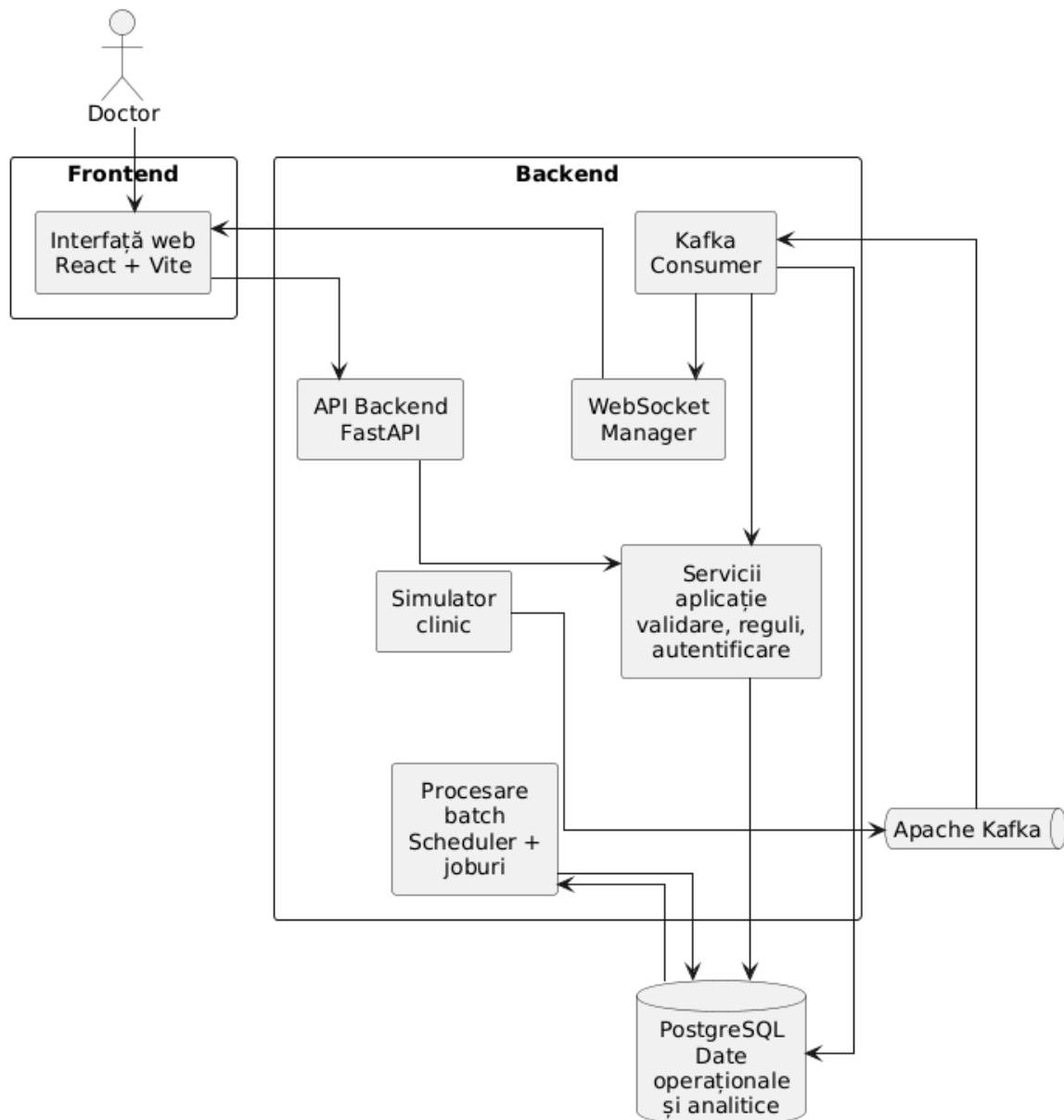


Figura 4.1: Arhitectura generală a aplicației MedStream

Figura 4.1 prezintă componentele principale ale aplicației și modul în care acestea comunică. Frontend-ul trimite cereri REST către backend pentru operațiile obișnuite și primește actualizări live prin WebSocket. Backend-ul coordonează logica aplicației, salvează datele în PostgreSQL, consumă evenimente din Kafka și pornește procesele de fundal necesare. Simulatorul generează valori vitale și alerte, Kafka transportă evenimentele recente, iar mecanismul batch calculează periodic rezultate agregate pe baza datelor persistate.

Din punct de vedere al procesării datelor, MedStream poate fi considerată o

aplicație hibridă, deoarece folosește în același sistem atât procesare streaming, cât și procesare batch. Cele două abordări nu sunt folosite separat, ci sunt integrate pentru a demonstra roluri diferite ale aceleiași arhitecturi. Streaming-ul este folosit pentru evenimente recente, monitorizare live și generarea alertelor, în timp ce batch-ul este folosit pentru agregări, metrici și analize periodice pe baza datelor acumulate. Această combinație evidențiază faptul că cele două tehnologii se pot completa într-o aplicație modernă de date, în loc să fie tratate ca soluții opuse.

4.1.1 Cazuri de utilizare

Diagrama cazurilor de utilizare prezintă fluxul principal al aplicației din perspectiva doctorului. Utilizatorul poate alege între înregistrare și autentificare, iar după autentificare ajunge în dashboard. Din dashboard se pot accesa lista departamentelor, lista alertelor, paginile de metrici, profilul doctorului și funcționalitatea de adăugare a unui pacient.

Din lista departamentelor și din lista alertelor se poate ajunge la profilul pacientului. În profilul pacientului sunt disponibile opțiuni precum editarea datelor pacientului și accesarea istoricului clinic. Paginile de metrici grupează vizualizările pentru streaming, batch și comparația dintre cele două abordări. Din profilul doctorului pot fi realizate acțiuni precum editarea profilului, ștergerea contului, asignarea sau eliminarea pacienților și adăugarea unei activități medicale.

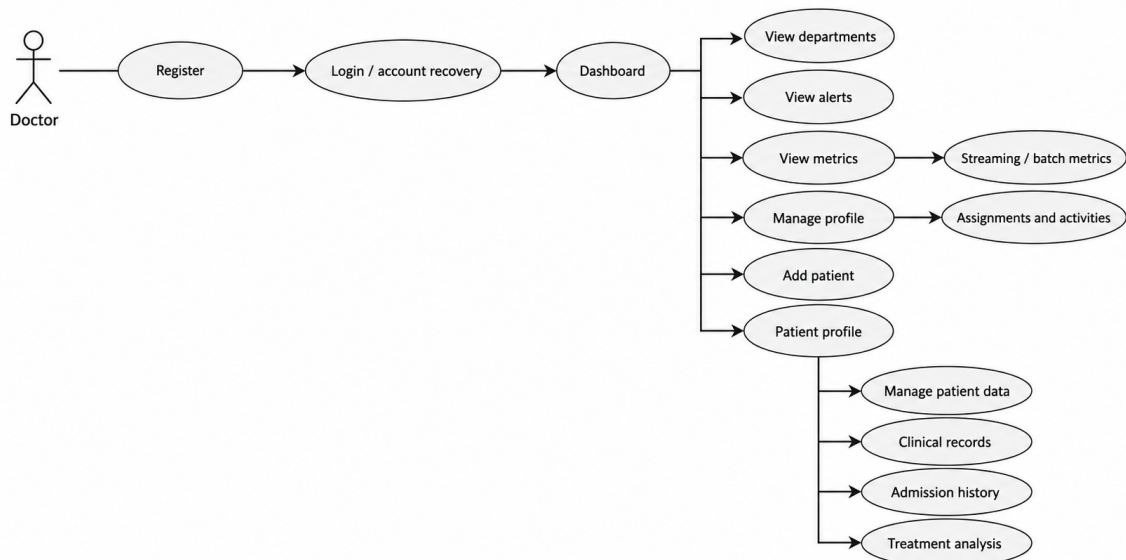


Figura 4.2: Diagrama cazurilor de utilizare pentru aplicația MedStream

4.1.2 Fluxul de procesare a alertelor

Fluxul de procesare a alertelor este unul dintre cele mai importante mecanisme ale aplicației, deoarece evidențiază modul în care datele generate în timp real sunt transformate în informații utile pentru doctor. Procesul începe cu generarea unei valori vitale pentru un pacient, continuă cu transmiterea evenimentului prin Kafka, procesarea acestuia în backend, evaluarea stării pacientului și, dacă este cazul, generarea unei alerte.

Evaluarea pacientului se face pe baza valorilor vitale precum pulsul, saturația de oxigen și temperatura. Sistemul analizează aceste valori folosind reguli explicite, nu modele de machine learning. Dacă valorile sunt normale, pacientul rămâne într-o stare stabilă și interfața primește doar o actualizare live. Dacă sunt detectate valori anormale, sistemul determină simptomele, actualizează starea pacientului, creează alerta și transmite evenimentul către interfață prin WebSocket.

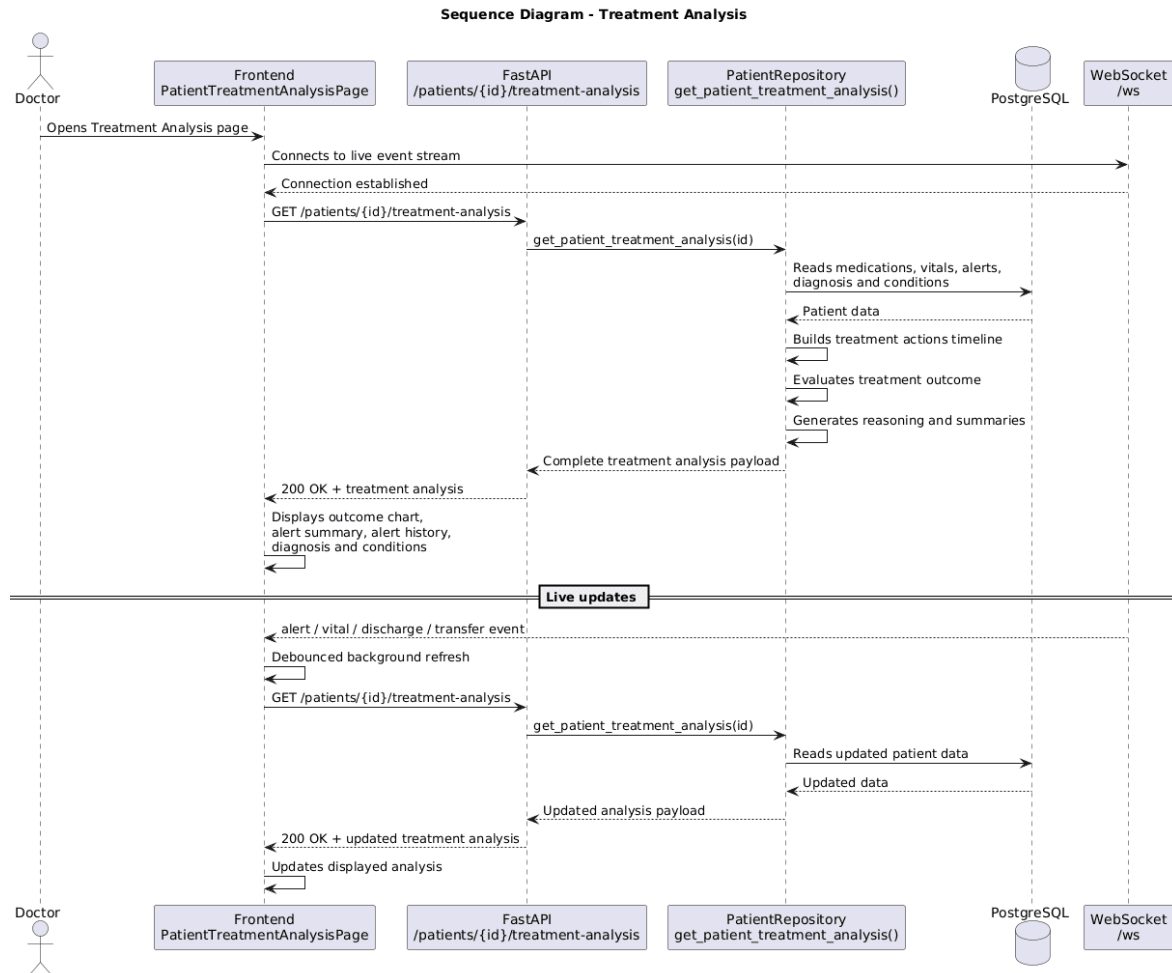


Figura 4.3: Diagrama de secvență pentru procesarea valorilor vitale

4.1.3 Modelul bazei de date

Modelul bazei de date este construit în jurul entităților principale ale aplicației: doctori, pacienți, valori vitale, alerte, activități medicale și informații clinice asociate fiecărui pacient. Baza de date are rolul de a păstra atât datele operaționale, folosite în fluxurile curente ale aplicației, cât și datele analitice rezultate din procesarea batch. Prin această organizare, sistemul poate susține simultan monitorizarea live și analiza periodică a datelor acumulate.

Pentru lizibilitate, modelul bazei de date este prezentat în mai multe subdiagramme. Această împărțire permite evidențierea separată a entităților principale, a tabelelor folosite pentru monitorizare, a datelor clinice și a rezultatelor analitice generate de sistem.

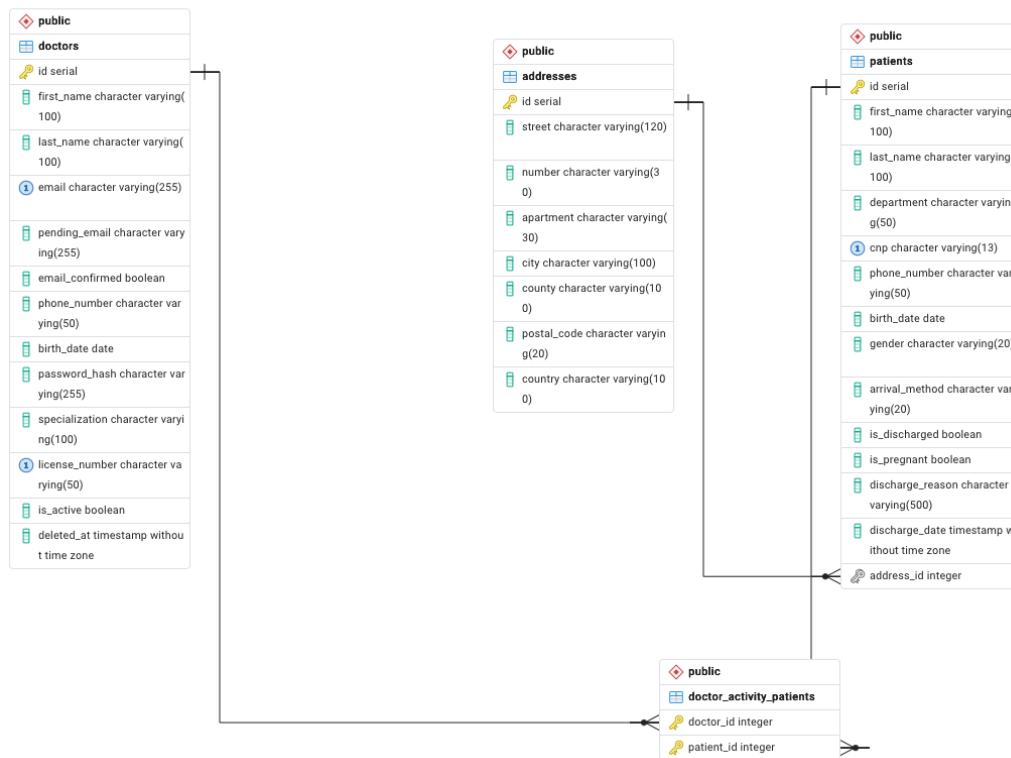


Figura 4.4: Modelul bazei de date: doctori, pacienți și asignări

Figura 4.4 prezintă entitățile de bază ale aplicației. Tabela `patients` reprezintă entitatea centrală a modelului, deoarece majoritatea informațiilor sunt legate direct sau indirect de pacient. Tabela `doctors` păstrează informațiile despre utilizatorii aplicației, inclusiv datele necesare autentificării și identificării doctorului în sistem. Relația dintre doctori și pacienți permite asignarea pacienților către doctori, astfel încât fiecare utilizator autentificat să poată gestiona pacienții asociați.

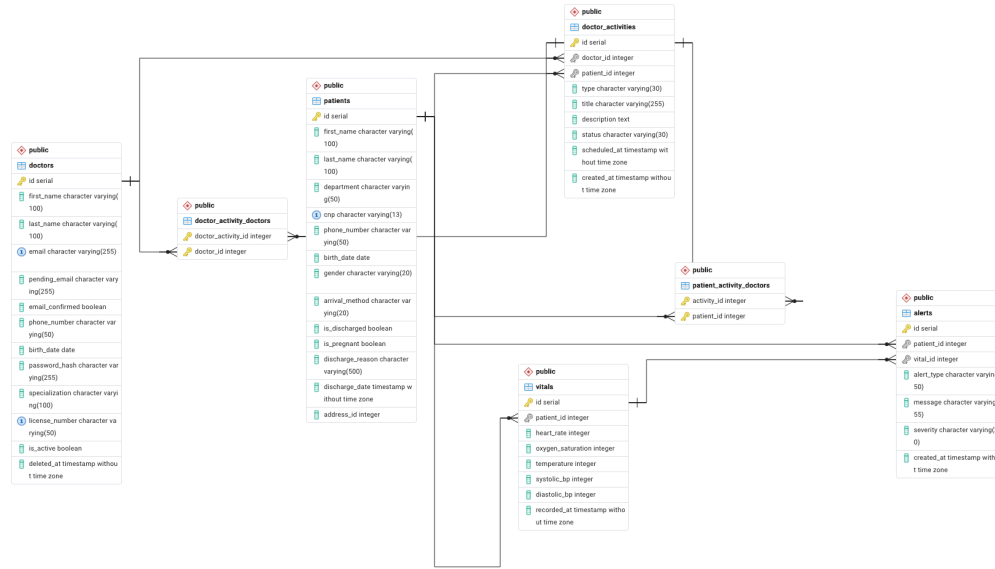


Figura 4.5: Modelul bazei de date: valori vitale, alerte și activități medicale

Figura 4.5 evidențiază tabelele folosite pentru monitorizarea pacienților și pentru generarea alertelor. Valorile vitale sunt asociate pacienților și includ date precum pulsul, nivelul de oxigen, temperatura sau tensiunea arterială. Alerte sunt create atunci când valorile generate depășesc pragurile definite în aplicație. Activitățile medicale sunt păstrate separat și permit reprezentarea unor acțiuni operaționale, cum ar fi activități noi, finalizate sau anulate. Aceste tabele sunt importante pentru fluxul streaming, deoarece reflectă datele recente produse de simulator și procesate de backend.

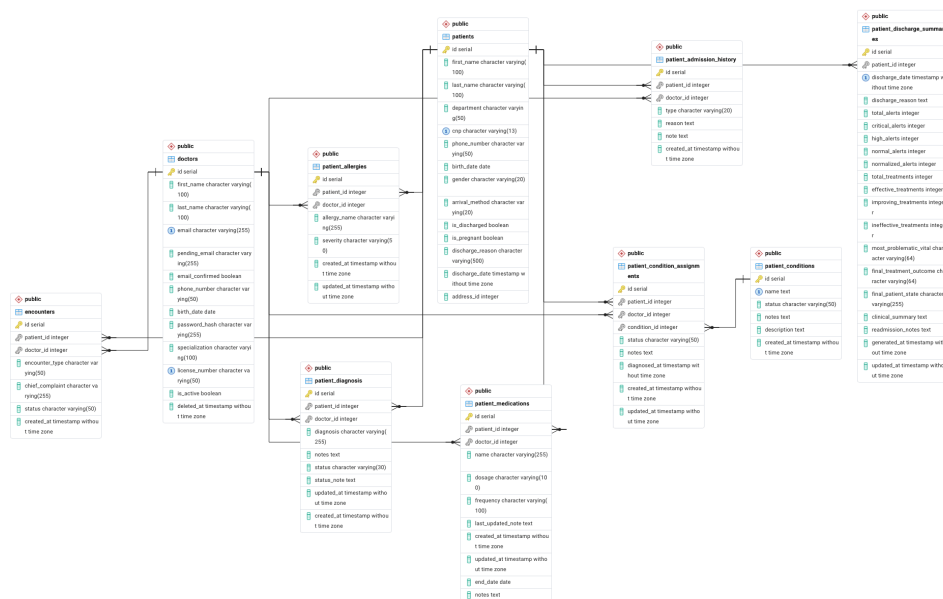


Figura 4.6: Modelul bazei de date: istoric clinic, diagnostice, alergii și medicație

Figura 4.6 grupează tabelele care descriu contextul clinic al pacientului. Diagnosticale descriu boli sau suspiciuni medicale, condițiile medicale descriu stări urmărite în timp, alergiile indică riscuri relevante, iar medicațiile indică tratamentele administrate. Separarea acestor informații în tabele dedicate permite extinderea istoricului clinic fără încărcarea excesivă a tabelului `patients`. În plus, aceste date oferă context pentru interpretarea alertelor și pentru analiza tratamentului.

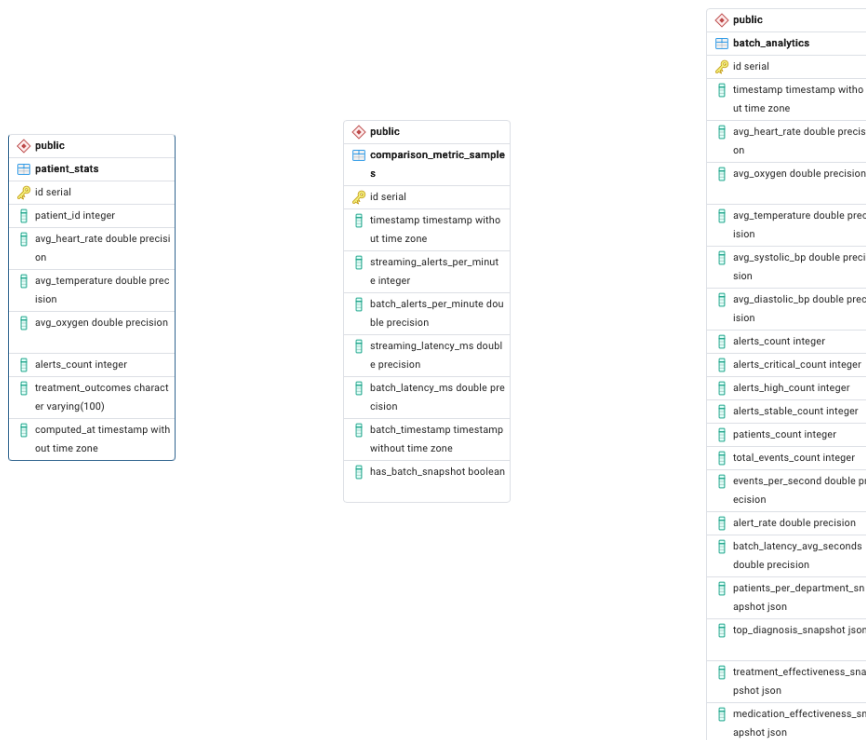


Figura 4.7: Modelul bazei de date: statistici, metrici batch și rezultate analitice

Figura 4.7 prezintă tabelele folosite pentru păstrarea rezultatelor analitice. Spre deosebire de valorile vitale și alertele individuale, aceste tabele conțin rezultate calculate periodic, precum statistici, snapshot-uri batch, metrici agregate și rezultate retrospective după externarea pacientului. Aceste date sunt folosite în paginile de metrici batch, în comparația dintre streaming și batch și în analiza tratamentelor.

Modelul relațional este potrivit pentru acest sistem deoarece permite definirea unor legături clare între entități și menținerea integrității datelor. Relațiile dintre tabele ajută la asocierea corectă a valorilor vitale, alertelor, activităților, tratamentelor și istoricului clinic cu pacientul corespunzător. În același timp, separarea dintre datele operaționale și datele analitice susține caracterul hibrid al aplicației, în care streaming-ul este folosit pentru evenimente recente, iar batch-ul pentru rezultate agregate și analize retrospective.

4.2 Implementarea sistemului

4.2.1 Configurarea mediului de rulare

4.2.1.1 Configurarea serverului

Pentru rularea locală a aplicației sunt necesare Python, Node.js și Docker. Docker Compose pornește serviciile auxiliare folosite de aplicație: frontend, backend, PostgreSQL, pgAdmin, Kafka, Zookeeper, Kafka UI și MailCatcher. În mediul de dezvoltare, aceste componente permit rularea completă a sistemului într-un mod controlat și apropiat de arhitectura finală a aplicației.

Comanda principală pentru pornirea aplicației și reconstruirea imaginilor Docker este:

```
docker compose up -d
```

Această comandă nu are doar rol de pornire a aplicației, ci permite testarea întregului flux demonstrativ într-un singur mediu local. Prin pornirea simultană a serviciilor, se poate verifica modul în care frontend-ul, backend-ul, baza de date, Kafka și componentele auxiliare colaborează fără configurarea manuală a fiecărei părți.

Backend-ul poate fi rulat și separat în timpul dezvoltării, prin crearea unui mediu virtual Python, instalarea dependențelor și pornirea serverului FastAPI:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt
uvicorn backend.app.main:app --reload
```

Rularea separată a backend-ului este utilă în timpul dezvoltării, deoarece permite observarea rapidă a mesajelor de log, a erorilor și a comportamentului serviciilor de fundal. În acest mod pot fi testate mai ușor componente precum simulatorul clinic, consumer-ul Kafka și scheduler-ul batch.

La pornirea backend-ului se inițializează baza de date, se verifică topicurile Kafka, se pornește consumer-ul Kafka, se pornește simulatorul clinic și se activează scheduler-ul pentru procesarea batch. Backend-ul rulează la adresa `http://localhost:8000`, iar starea serviciului poate fi verificată prin endpoint-ul `/health`. Acest endpoint este util pentru verificarea rapidă a faptului că serverul răspunde corect înainte de testarea fluxurilor din interfață.

Interfețele auxiliare sunt disponibile local la `http://localhost:5050` pentru pgAdmin, `http://localhost:8080` pentru Kafka UI și `http://localhost:1080` pentru MailCatcher. Aceste instrumente nu fac parte din funcționalitatea

principală a aplicației, dar ajută la verificarea bazei de date, a topicurilor Kafka și a fluxurilor de email în timpul dezvoltării.

4.2.1.2 Configurarea bazei de date

Baza de date PostgreSQL este pornită prin Docker Compose și folosește variabile de mediu pentru numele bazei de date, utilizator, parolă și URL-ul de conexiune. În configurația locală, aplicația folosește o bază de date dedicată pentru MedStream, iar conexiunea este realizată de backend prin SQLAlchemy.

La pornirea backend-ului, aplicația execută verificări de inițializare pentru structura bazei de date. Aceste verificări creează sau completează elementele necesare pentru rularea locală a sistemului. Pentru un mediu de producție, o variantă mai potrivită ar fi utilizarea unor migrații versionate, însă pentru contextul demonstrativ al aplicației inițializarea la startup este suficientă.

Datele demonstrative nu sunt populate prin module seed separate, ci prin simulatorul aplicației. La pornirea backend-ului, simulatorul este lansat automat ca proces de fundal, creează doctori și pacienți demonstrativi atunci când este necesar și generează periodic valori vitale, alerte, activități și evoluții clinice. Astfel, aplicația poate fi testată fără rularea manuală a unor scripturi de seed dedicate.

4.2.1.3 Configurarea frontend-ului

Frontend-ul se pornește din directorul `frontend`. După instalarea dependențelor, aplicația React este rulată prin serverul de dezvoltare Vite:

```
cd frontend
npm install
npm run dev
```

Interfața comunică cu backend-ul prin cereri REST și prin WebSocket. Această combinație permite încărcarea datelor persistente prin API și actualizarea live a anumitor pagini atunci când apar evenimente noi.

4.2.2 Implementarea backend

4.2.2.1 Sistemul de autentificare

Sistemul de autentificare permite doctorilor să își creeze cont și să se conecteze în aplicație. Fluxul începe cu pagina de register, continuă cu login-ul, iar după autentificare utilizatorul ajunge în dashboard. Autentificarea este importantă deoarece funcționalitățile interne ale aplicației trebuie asociate unui doctor autentificat.

Parolele sunt tratate prin hashing, iar backend-ul validează datele primite înainte de procesare. Pe lângă login și register, aplicația include și fluxuri de suport precum verificarea email-ului, recuperarea contului și resetarea parolei. În mediul local, aceste fluxuri pot fi testate cu ajutorul MailCatcher.

4.2.2.2 Rute API

Backend-ul expune rute REST pentru principalele zone ale aplicației: doctori, pacienți, valori vitale, alerte, statistici, metrice și procesare batch. Aceste rute sunt consumate de frontend pentru afișarea datelor și pentru executarea acțiunilor utilizatorului.

Rutele pentru pacienți gestionează operații precum listarea, adăugarea, editarea, transferul, externarea și readmiterea. Rutele pentru alerte expun alertele globale și alertele asociate unui pacient. Rutele pentru metrice expun datele necesare paginilor de streaming, batch și comparison.

4.2.2.3 Validarea datelor și repository-uri

Validarea datelor este realizată cu Pydantic, iar accesul la baza de date este gestionat prin SQLAlchemy. Această separare permite ca datele primite prin API să fie verificate înainte de a ajunge în logica aplicației sau în baza de date.

Repository-urile au rolul de a izola operațiile de citire și scriere asupra bazei de date. Prin această abordare, rutele API nu trebuie să conțină direct interogări complexe, iar logica aplicației rămâne mai ușor de urmărit și de modificat.

4.2.2.4 Servicii streaming și WebSocket

Backend-ul include servicii de fundal pentru consumul evenimentelor Kafka și pentru transmiterea actualizărilor live către frontend. Evenimentele relevante sunt preluate din topicurile Kafka, procesate și apoi transmise prin WebSocket către paginile care au nevoie de actualizări în timp real.

Topicurile `vitals-events` și `alerts-events` sunt consumate direct de backend pentru actualizarea bazei de date, a metricilor streaming și a interfeței prin WebSocket. Topic-ul `batch-events` există în infrastructură și poate fi observat în Kafka UI, însă metricile batch afișate în aplicație sunt calculate din datele persistate în PostgreSQL și salvate ca snapshot-uri analitice.

Această separare este intenționată. Kafka este folosit pentru evenimente recente și pentru demonstrarea fluxului streaming, în timp ce PostgreSQL este folosit ca sursă stabilă pentru calculele batch. Batch-ul nu calculează direct din Kafka deoarece scopul său este analiza datelor deja acumulate și persistate, nu reacția ime-

diată la fiecare eveniment. Astfel, streaming-ul răspunde la schimbările curente, iar batch-ul folosește istoricul salvat pentru rezultate agregate.

WebSocket este folosit pentru evenimente precum valori vitale noi, alerte, transferuri sau externări. Totuși, nu toate paginile aplicației folosesc WebSocket. Paginile care afișează date istorice, formulare sau rezultate stabile pot folosi cereri REST obișnuite, deoarece nu necesită actualizare imediată. WebSocket este rezervat zonelor unde schimbările trebuie să fie vizibile rapid, cum ar fi dashboard-ul, pagina de alerte și profilul pacientului.

4.2.2.5 Controller-ul de procesare batch

Procesarea batch este coordonată de un controller dedicat și de un scheduler. Joburile batch pot fi rulate periodic sau manual, iar rezultatele sunt salvate în baza de date pentru a fi consumate ulterior de frontend.

În timpul unei rulări batch, sistemul calculează statistici per pacient, agregări pe departamente, distribuții ale alertelor și informații legate de tratamente. Rezultatele sunt persistate în tabelele dedicate și sunt expuse prin endpoint-uri precum `/metrics/batch`, `/metrics/batch-insights` și `/metrics/comparison`.

Persistarea rezultatelor batch este importantă deoarece permite păstrarea unor snapshot-uri stabile ale datelor analizate. Spre deosebire de streaming, unde valorile se pot modifica rapid pe măsură ce apar evenimente noi, batch-ul oferă o imagine calculată la un anumit moment. Această diferență susține scopul lucrării, deoarece face vizibil contrastul dintre actualizările live și rezultatele periodice.

4.2.3 Implementarea frontend

4.2.3.1 Rutare și pagini protejate

Frontend-ul aplicației este dezvoltat în React și Vite, folosind React Router pentru navigare și Cloudscape Design Components pentru componentele principale de interfață. Rutele aplicației sunt definite în `frontend/src/App.jsx`, iar punctul de intrare al aplicației este `frontend/src/main.jsx`. Interfața este organizată în pagini și componente reutilizabile, iar stilizarea specifică este completată prin fișiere CSS separate, asociate paginilor sau zonelor funcționale.

Paginile interne sunt accesibile după autentificare. Această organizare reflectă fluxul principal al aplicației: utilizatorul se autentifică, ajunge în dashboard și de acolo poate accesa zonele funcționale.

4.2.3.2 Navigation bar și layout

Navigarea în aplicație este organizată printr-un layout autentificat, care include bară superioară, navigație laterală, icon rail atunci când meniul este restrâns și o zonă principală de conținut. Bara de navigare permite accesul rapid la dashboard, departamente, alerte, metrice și profil. Titlul documentului se actualizează în funcție de pagina accesată, iar pentru paginile de pacient aplicația poate afișa numele pacientului în titlul paginii.

Layout-ul are și rolul de a păstra o structură vizuală consistentă între pagini. Cloudscape oferă baza pentru componentele de navigare și organizare vizuală, iar fișierele CSS personalizate permit ajustarea designului pentru cerințele specifice aplicației MedStream. Astfel, chiar dacă fiecare pagină are un scop diferit, utilizatorul păstrează același mod de navigare și aceeași logică de interacțiune.

4.2.3.3 Serviciul API și clientul WebSocket

Comunicarea cu backend-ul este separată în servicii dedicate. Serviciul API este folosit pentru cererile REST, iar serviciul WebSocket gestionează conexiunea live cu backend-ul. Această separare evită duplicarea logicii de comunicare în fiecare pagină.

Paginile care afișează date stabile sau istorice folosesc în principal cereri REST, în timp ce paginile care au nevoie de actualizări rapide folosesc WebSocket. De exemplu, dashboard-ul, pagina de alerte și profilul pacientului pot primi actualizări live atunci când apar valori vitale sau alerte noi.

4.2.3.4 Componente reutilizabile și grafice

Frontend-ul folosește componente reutilizabile pentru grafice, tabele, carduri, formulare și zone de conținut. O parte dintre componentele vizuale sunt construite cu Cloudscape Design Components, iar elementele specifice aplicației sunt ajustate prin CSS personalizat. Această abordare permite păstrarea unei interfețe coerente, dar suficient de flexibilă pentru pagini precum dashboard, profil pacient, alerte și metrice.

Pentru vizualizarea datelor este utilizată biblioteca Recharts. Aceasta permite construirea graficelor de tip linie, bară, arie sau pie chart, folosite în paginile de dashboard, pacient, streaming metrics, batch metrics și treatment analysis.

4.2.3.5 Pagini clinice

Paginile clinice sunt cele care oferă contextul medical al aplicației. Profilul pacientului prezintă datele generale, valorile vitale recente și alertele asociate, iar istoricul

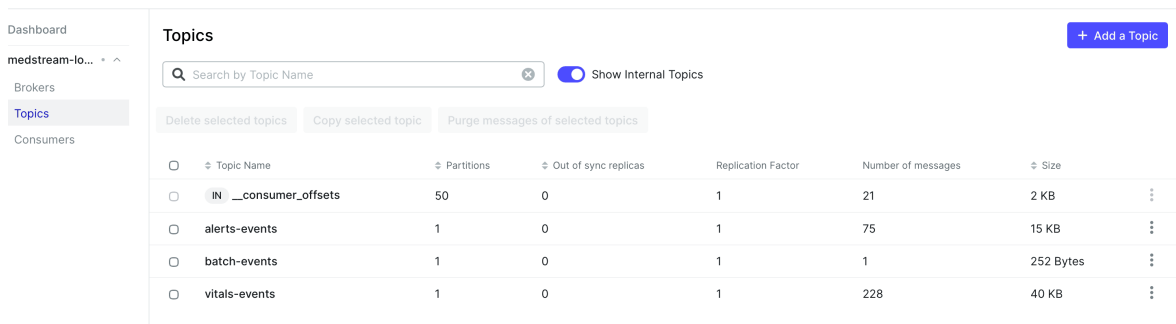
clinic include diagnostice, alergii, condiții și medicație.

Aceste pagini nu sunt neapărat streaming sau batch în mod direct, dar oferă contextul necesar pentru interpretarea alertelor și a analizelor. Ele susțin partea demonstrativă a aplicației, deoarece transformă fluxurile de date într-un scenariu mai realist.

4.2.4 Instrumente auxiliare pentru dezvoltare

4.2.4.1 Monitorizarea topicurilor Kafka

Kafka UI este inclus în infrastructura locală pentru verificarea topicurilor Kafka și a mesajelor produse de aplicație. În MedStream, topicurile relevante sunt `vitals-events`, `alerts-events` și `batch-events`. Interfața Kafka UI permite observarea mesajelor, a topicurilor și a stării consumatorilor.



The screenshot shows the Kafka UI 'Topics' page. On the left is a sidebar with 'Dashboard', 'medstream-lo...', 'Brokers', 'Topics' (selected), and 'Consumers'. The main area has a search bar, a 'Show Internal Topics' toggle, and buttons for 'Delete selected topics', 'Copy selected topic', and 'Purge messages of selected topics'. Below is a table of topics.

☐	Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size	
☐	IN __consumer_offsets	50	0	1	21	2 KB	⋮
☐	alerts-events	1	0	1	75	15 KB	⋮
☐	batch-events	1	0	1	1	252 Bytes	⋮
☐	vitals-events	1	0	1	228	40 KB	⋮

Figura 4.8: Vizualizarea topicurilor Kafka în Kafka UI

Această componentă este utilă în timpul dezvoltării deoarece permite verificarea fluxului dintre simulator, Kafka și backend. Dacă o alertă nu apare în interfață, Kafka UI poate fi folosit pentru a verifica dacă evenimentul a fost produs corect în topic. În același timp, este important de menționat că backend-ul consumă direct doar topicurile de vitale și alerte, în timp ce rezultatele batch sunt calculate din baza de date și din snapshot-urile generate periodic.

4.2.4.2 Testarea fluxurilor de email

MailCatcher este folosit în mediul local pentru testarea funcționalităților care implică email-uri. Acesta interceptează mesajele generate de aplicație și le afișează într-o interfață web locală, fără ca mesajele să fie trimise către adrese reale.

În MedStream, MailCatcher este util pentru fluxuri precum verificarea email-ului, recuperarea contului sau resetarea parolei. Astfel, funcționalitățile de autentificare pot fi testate în siguranță, fără dependența de un furnizor extern de email.

4.3 Simulatorul de date și integrarea fluxurilor

Integrarea fluxurilor din MedStream urmărește să evidențieze caracterul hibrid al aplicației. Aceleași date generate de simulator pot produce actualizări live prin streaming și pot fi salvate ulterior pentru calcule batch. În acest fel, aplicația demonstrează că procesarea imediată și procesarea periodică pot funcționa împreună, folosind același context și aceleași tipuri de date.

4.3.1 Rolul simulatorului

Simulatorul este componenta care generează datele clinice sintetice folosite de aplicație. Acesta creează doctori, pacienți, valori vitale, alerte, tratamente și evoluții clinice controlate. Scopul său este de a produce un flux constant de date, astfel încât aplicația să poată demonstra diferența dintre procesarea streaming și procesarea batch.

Simulatorul pornește automat la inițializarea backend-ului pentru ca aplicația să poată fi testată imediat, fără pregătirea manuală a unui set mare de date. În contextul lucrării, acest mecanism este important deoarece permite generarea continuă de date fără intervenția utilizatorului, ceea ce face posibilă observarea diferenței dintre actualizările live și rezultatele batch.

Datele generate nu reprezintă cazuri medicale reale, ci scenarii demonstrative. Simulatorul poate produce pacienți stabili, pacienți în stare de avertizare și pacienți în stare critică. În funcție de valorile vitale, sistemul poate crea alerte, poate modifica starea pacientului și poate pregăti datele necesare pentru analiza tratamentului.

4.3.2 Generarea valorilor vitale și a alertelor

Valorile vitale urmărite sunt pulsul, saturația de oxigen, temperatura și tensiunea arterială. Aceste valori sunt generate periodic și sunt asociate pacienților activi. Pragurile folosite în aplicație sunt definite explicit în cod și permit clasificarea stării pacientului.

Alertele sunt generate atunci când anumite valori vitale trec prin schimbări relevante. Pulsul peste 110 bpm este tratat ca valoare ridicată, iar peste 130 bpm poate fi considerat critic. Oxigenul sub 92% indică o problemă, iar sub 88% poate genera o alertă critică. Temperatura peste 38°C poate produce o alertă de tip high, iar peste 39°C poate produce o alertă critică. Atunci când valorile revin în limite normale, sistemul poate genera alerte de tip *normalized*.

Tensiunea arterială este generată, salvată și folosită în agregările batch, însă în logica actuală nu declanșează alerte. Alertele sunt determinate de tranzițiile de stare pentru puls, saturația de oxigen și temperatură.

4.3.3 Pregătirea datelor pentru analiza tratamentului

Simulatorul nu generează doar valori vitale izolate, ci și date necesare pentru analiza tratamentelor. Pe măsură ce pacientul evoluează, aplicația poate înregistra tratamente, alerte, valori vitale și schimbări de stare. Aceste informații sunt folosite ulterior în pagina Treatment Analysis pentru a evalua dacă tratamentul a fost *Effective*, *Improving* sau *Ineffective*.

Logica este rule-based, nu bazată pe machine learning. Sistemul compară starea pacientului înainte și după tratament, verifică alertele recente și evaluează dacă valorile vitale indică stabilizare, îmbunătățire parțială sau lipsa unui răspuns favorabil. Această abordare face analiza explicabilă și potrivită pentru o aplicație demonstrativă.

4.3.4 Integrarea cu procesarea streaming

Fluxul streaming intervine atunci când evenimentele generate sunt publicate în Kafka și procesate de backend. Topicurile consumate direct de pipeline-ul principal sunt `vitals-events` și `alerts-events`. Consumer-ul Kafka preia aceste evenimente, actualizează store-ul de metrice streaming și transmite datele relevante către frontend prin WebSocket.

Prin acest mecanism, dashboard-ul, pagina de alerte, profilul pacientului și analiza tratamentului pot primi actualizări recente fără reîncărcarea manuală a paginii. Streaming-ul este folosit pentru întrebarea „ce se întâmplă acum?” și pentru afișarea rapidă a alertelor și schimbărilor de stare.

4.3.5 Integrarea cu procesarea batch

Procesarea batch folosește datele acumulate în baza de date pentru a calcula rezultate agregate. Jobul batch rulează periodic sau manual și produce statistici precum medii ale valorilor vitale, număr de alerte, pacienți activi, evenimente procesate, rată de alerte, distribuții pe departamente și eficiența tratamentelor.

Alegerea PostgreSQL ca sursă pentru batch este potrivită deoarece baza de date conține istoricul stabil al evenimentelor și al entităților aplicației. Kafka este folosit pentru transportul evenimentelor recente, dar batch-ul are nevoie de o imagine coerentă asupra datelor salvate, care poate fi interogată și agregată la momentul rulării. Astfel, procesarea batch reflectă datele persistate, nu doar evenimentele aflate în tranzit.

Rezultatele batch sunt salvate ca snapshot-uri și sunt afișate în paginile Batch Metrics și Comparison. Batch-ul este folosit pentru întrebarea „ce s-a întâmplat într-o perioadă?” și oferă o perspectivă mai stabilă asupra datelor decât fluxul streaming.

Implementarea descrisă evidențiază modul în care componentele aplicației colaborează pentru a susține fluxul complet al datelor: generare prin simulator, transmitere prin Kafka, procesare în backend, persistență în PostgreSQL și afișare în interfață. Separarea dintre actualizările live și calculele batch permite urmărirea celor două paradigme în același context aplicativ.

După stabilirea modului în care sistemul este construit, analiza se poate concentra asupra utilizării aplicației, a funcționalităților disponibile în interfață și a rezultatelor generate în timpul rulării.

Capitolul 5

Scenarii de utilizare și rezultatele obținute

Utilizarea aplicației MedStream evidențiază modul în care arhitectura implementată se reflectă în interfață și în rezultatele generate. Funcționalitățile disponibile pentru doctor, procesele automate ale sistemului, metricile streaming și snapshot-urile batch oferă perspectiva practică asupra diferenței dintre procesarea imediată și procesarea periodică.

Rezultatele prezentate în această parte provin din rularea aplicației cu date clinice sintetice. Ele nu au rolul unui benchmark general pentru tehnologii de procesare a datelor, ci susțin demonstrația urmărită în lucrare: aceeași aplicație poate folosi streaming pentru reacție rapidă și batch pentru analiză agregată.

5.1 Rezultatele generate și metricile aplicației

Rezultatele generate de aplicație au fost analizate pe baza metricilor exportate din pagina de comparație dintre procesarea streaming și procesarea batch. Valorile provin din rularea aplicației cu date clinice sintetice și reflectă comportamentul sistemului într-un scenariu demonstrativ. Ele nu trebuie interpretate ca un benchmark general pentru tehnologiile utilizate, ci ca dovezi ale diferenței dintre actualizarea live a datelor și procesarea periodică a acestora.

5.1.1 Metrici generale

În snapshot-ul analizat, aplicația a procesat un total de 38.460 de evenimente și a generat 4.386 de alerte. Rata alertelor a fost de 0,114, ceea ce înseamnă că aproximativ 11,4% dintre evenimente au produs sau au fost asociate cu o alertă. De asemenea, rata de procesare a fost de aproximativ 10,68 evenimente pe secundă.

Aceste valori arată că sistemul nu funcționează doar pe câteva exemple izolate, ci pe un flux continuu de date simulate. Numărul total de evenimente oferă context pentru analiza procesării, iar numărul total de alerte permite observarea diferenței dintre alertele recente, tratate prin streaming, și valorile agregate, calculate prin batch.

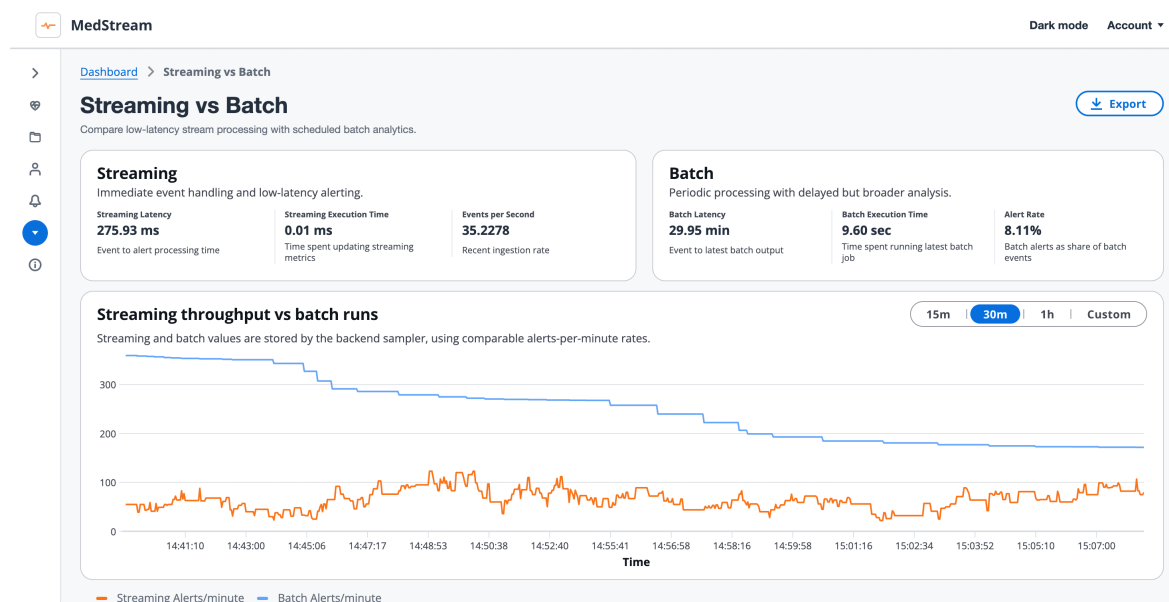


Figura 5.1: Metrice generate în pagina de comparație dintre streaming și batch

5.1.2 Latența streaming și batch

Una dintre diferențele urmărite în aplicație este latența. În datele exportate, latența medie pentru batch este de 1.780.450 ms, adică aproximativ 29,67 minute. Această valoare reflectă natura procesării batch, unde rezultatele nu sunt actualizate la fiecare eveniment individual, ci după o rulare periodică sau după calcularea unui snapshot agregat.

Pentru streaming, valoarea exportată a latenței medii este 0 ms, iar câmpul asociat timestamp-ului streaming este gol în snapshot-ul analizat. Din acest motiv, valoarea nu trebuie interpretată ca o latență tehnică reală de zero, ci ca o valoare raportată de exportul curent. Totuși, comportamentul aplicației arată diferența urmărită: streaming-ul actualizează datele incremental, în timp ce batch-ul produce rezultate agregate cu o întârziere mai mare.

În același export, diferența de latență este egală cu 1.780.450 ms, deoarece valoarea streaming este raportată ca 0 ms. Interpretarea importantă nu este valoarea absolută a latenței streaming, ci faptul că batch-ul introduce un decalaj vizibil între momentul apariției datelor și momentul disponibilității rezultatului agregat.

5.1.3 Evoluția alertelor

Evoluția alertelor a fost analizată pe baza valorilor exportate pentru câmpurile `streaming_alerts_per_minute` și `batch_alerts_per_minute`. Fragmentul analizat acoperă intervalul 08:56:04–08:56:20 și include două snapshot-uri batch, generate la 08:53:34 și 08:55:49. Valorile sunt prezentate în Tabelul 5.1.

Ora exportului	Streaming	Batch	Diferență	Raport	Snapshot batch
08:56:04.637	0	159,65	159,65	–	08:53:34.620
08:56:05.388	0	159,65	159,65	–	08:53:34.620
08:56:08.592	14	159,65	145,65	11,4036	08:53:34.620
08:56:09.500	14	159,65	145,65	11,4036	08:53:34.620
08:56:12.594	14	160,9833	146,9833	11,4988	08:55:49.662
08:56:13.618	14	160,9833	146,9833	11,4988	08:55:49.662
08:56:16.598	19	160,9833	141,9833	8,4728	08:55:49.662
08:56:17.654	19	160,9833	141,9833	8,4728	08:55:49.662
08:56:20.600	19	160,9833	141,9833	8,4728	08:55:49.662

Tabela 5.1: Evoluția alertelor pe minut în fereastra streaming și în snapshot-ul batch

În primele două înregistrări, valoarea streaming este 0, deoarece fereastra live nu raportează încă alerte în momentul respectiv. Ulterior, valoarea streaming crește la 14 și apoi la 19 alerte pe minut, ceea ce indică actualizarea incrementală a fluxului live. În schimb, valoarea batch rămâne stabilă pentru mai multe înregistrări consecutive: inițial 159,65 alerte pe minut, apoi 160,9833 alerte pe minut după generarea unui nou snapshot batch.

Diferența dintre cele două valori scade de la 145,65 la 141,9833 atunci când fereastra streaming acumulează mai multe alerte recente. Raportul batch/streaming scade de la aproximativ 11,49 la 8,47, ceea ce arată că valoarea streaming se modifică rapid în funcție de evenimentele curente, în timp ce valoarea batch rămâne legată de ultimul snapshot calculat.

Această evoluție evidențiază diferența dintre cele două paradigme. Streaming-ul reflectă schimbările recente și se modifică pe măsură ce apar evenimente noi, în timp ce batch-ul păstrează o valoare agregată până la următoarea execuție. Prin urmare, streaming-ul este potrivit pentru observarea stării curente a sistemului, iar batch-ul este potrivit pentru obținerea unei imagini agregate asupra unei perioade.

5.1.4 Interpretarea rezultatelor

Rezultatele obținute confirmă rolurile diferite ale celor două mecanisme de procesare. Componenta streaming este potrivită pentru actualizări rapide, monitorizare live și alerte recente, deoarece valorile se modifică incremental pe măsură ce apar evenimente noi. Componenta batch este potrivită pentru agregări, snapshot-uri și analize retrospective, deoarece rezultatele sunt calculate periodic și nu se modifică la fiecare eveniment individual.

În contextul MedStream, aceste diferențe sunt vizibile atât în metricile numerice, cât și în interfață. Paginile de streaming și alerte evidențiază reacția rapidă la evenimente, în timp ce pagina de metrice batch oferă o vedere mai stabilă asupra datelor acumulate. Pagina de comparație reunește aceste perspective și permite observarea diferenței dintre fluxul live și rezultatul periodic.

Rezultatele sunt relevante pentru scopul lucrării deoarece arată că aplicația nu compară cele două abordări doar la nivel teoretic. Evenimentele generate de simulator, alertele transmise live și snapshot-urile batch oferă exemple concrete pentru diferența dintre procesarea imediată și analiza periodică.

5.2 Raportarea la studii experimentale

Rezultatele obținute în MedStream nu pot fi comparate direct cu benchmark-uri industriale, deoarece aplicația folosește date clinice sintetice și metrice specifice scenariului implementat, precum alerte pe minut. Totuși, valorile pot fi raportate la comparații experimentale existente, pentru a verifica dacă tendințele observate sunt similare.

Sursă / sistem	Tip procesare	Rată / throughput	Timp / latență
MedStream – fereastră live	Streaming	0–19 alerte/minut	actualizare incrementală
MedStream – snapshot batch	Batch	159,65–160,98 alerte/minut	1.780.450 ms
Pacella et al. [PPPF25] – Spark Streaming	Streaming	612,25 records/s	4,236 s total; 1,390 s latență medie
Pacella et al. [PPPF25] – Hadoop Batch	Batch	1.673,45 records/s	5,976 s total; latență neraportată

Tabela 5.2: Comparație numerică între MedStream și studii experimentale

Tabelul 5.2 trebuie interpretat ca o raportare contextuală, nu ca un benchmark direct. MedStream folosește date clinice sintetice și raportează metrice specifice

aplicației, precum alerte pe minut, alerte per rulare batch și latența snapshot-ului batch. În schimb, studiile Pacella et al. raportează rezultate pentru date de senzori industriali, folosind metrici precum throughput în records/s, timp total de procesare și latență medie.

Din acest motiv, valorile nu sunt comparabile numeric în mod direct. Totuși, ele pot fi comparate la nivelul tendințelor observate. În studiul Pacella et al. [PPPF25], Hadoop Batch obține un throughput mai mare decât Spark Streaming, însă Spark Streaming oferă latență medie redusă și rezultate mai potrivite pentru scenarii care necesită reacție rapidă. În studiul Pacella et al. [PPP26], aceeași diferență este evidențiată prin faptul că Hadoop susține analiza retrospectivă cu throughput ridicat, în timp ce Spark Streaming oferă răspuns aproape în timp real pentru monitorizare continuă.

În MedStream, aceeași idee este ilustrată la scară demonstrativă. Componenta streaming actualizează fereastra live și alertele recente pe măsură ce apar evenimente noi, în timp ce componenta batch produce snapshot-uri agregate cu o întârziere mai mare. Astfel, aplicația nu urmărește să demonstreze performanța industrială, ci să evidențieze comportamentul diferența dintre actualizarea imediată și analiza periodică a datelor acumulate.

5.2.1 Compararea cu rezultate din surse existente

Rezultatele obținute în MedStream nu trebuie interpretate ca un benchmark general pentru procesarea streaming sau batch, deoarece aplicația folosește date simulate și rulează într-un mediu demonstrativ. Totuși, comportamentul observat este în acord cu diferențele descrise în studiile experimentale: streaming-ul este potrivit pentru rezultate actualizate rapid, iar batch-ul este potrivit pentru procesarea periodică a datelor acumulate.

În modelul Dataflow, procesarea fluxurilor nelimitate este discutată în raport cu latența, corectitudinea și costul procesării, în prezența evenimentelor întârziate sau sosite în altă ordine [ABC⁺15]. În MedStream, această idee este reflectată prin faptul că evenimentele recente sunt procesate incremental pentru actualizări live și alerte, în timp ce rezultatele batch sunt calculate separat, pe baza datelor persistate.

Abordări precum Spark Streaming și Apache Flink arată că în practică există mai multe forme intermediare între batch clasic și streaming continuu [ZDL⁺13]. MedStream nu urmărește să reproducă performanța acestor platforme, ci să demonstreze la scară redusă aceeași idee generală: procesarea imediată și procesarea periodică pot coexista în aceeași aplicație și pot răspunde unor nevoi diferite.

5.3 Funcționalități

5.3.1 Funcționalități utilizator

Funcționalitățile pentru doctor acoperă interacțiunea directă cu aplicația. Acestea includ autentificarea, administrarea profilului, gestionarea pacienților și completarea informațiilor clinice. Acțiunile sunt inițiate manual și sunt validate atât în interfață, cât și în backend.

5.3.1.1 Autentificare și profil doctor

Autentificarea în MedStream controlează accesul la funcționalitățile interne ale aplicației, precum dashboard-ul, profilul pacientului, istoricul clinic, alertele și paginile de metrici. Utilizatorul poate crea un cont prin pagina de register și se poate autentifica prin pagina de login. După autentificare, doctorul ajunge în dashboard, de unde poate accesa restul funcționalităților aplicației.

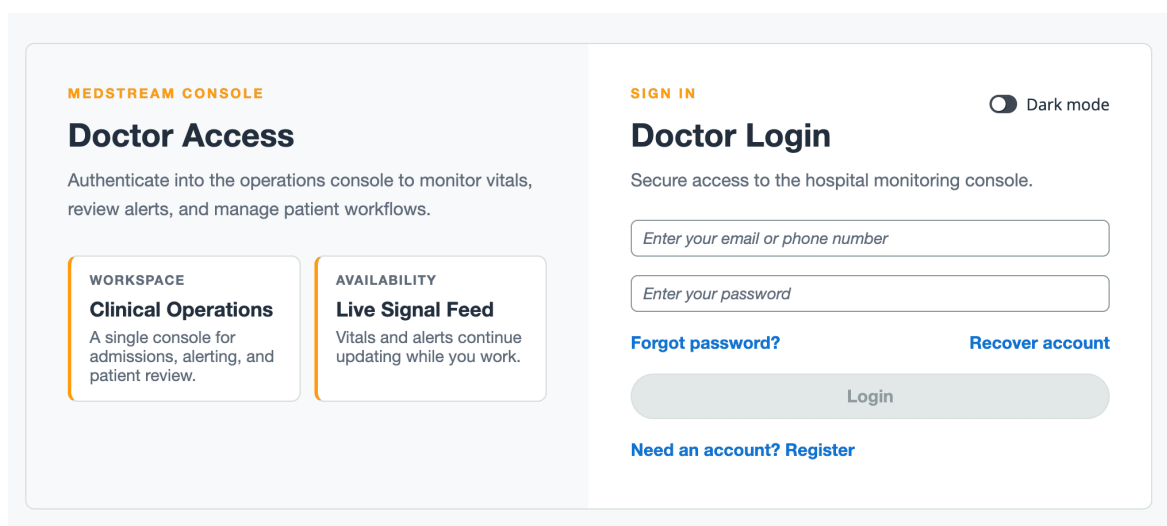


Figura 5.2: Pagina de înregistrare și autentificare în aplicația MedStream

- **Pagini implicate:** /register, /login, /forgot-password, /reset-password, /verify-email, /recover-account și /profile.
- **Date introduse la înregistrare:** nume, prenume, data nașterii, email, număr de telefon, specializare, număr de licență, parolă și confirmare parolă.
- **Validări principale:** email valid și unic, telefon unic atunci când este furnizat, parolă de minimum 8 caractere cu cel puțin o majusculă și o cifră, confirmare parolă, dată de naștere care nu poate fi în viitor, specializare validă și număr de licență unic.

- **Verificare email:** după înregistrare, contul este creat cu email neconfirmat, iar utilizatorul trebuie să acceseze link-ul primit pentru activare. Dacă link-ul expiră, aplicația permite retransmiterea email-ului de verificare.
- **Login:** autentificarea se face folosind email sau telefon și parolă. La autentificare reușită, backend-ul returnează un token de acces de tip Bearer, folosit ulterior pentru accesarea rutelor protejate.
- **Rute protejate:** paginile interne, precum dashboard-ul, profilul pacientului, pagina de alerte, metricile și istoricul clinic, sunt disponibile doar după autentificare.
- **Token expirat sau invalid:** frontend-ul elimină tokenul local, marchează utilizatorul ca neautentificat și îl redirecționează către pagina de login.
- **Funcționalități de suport:** aplicația include recuperare cont, resetare parolă și verificare email, utile pentru administrarea accesului utilizatorului.

Pagina de profil permite doctorului să își vizualizeze și să își editeze datele. Tot din această zonă pot fi gestionate acțiuni legate de pacienții asignați și de activitățile medicale.

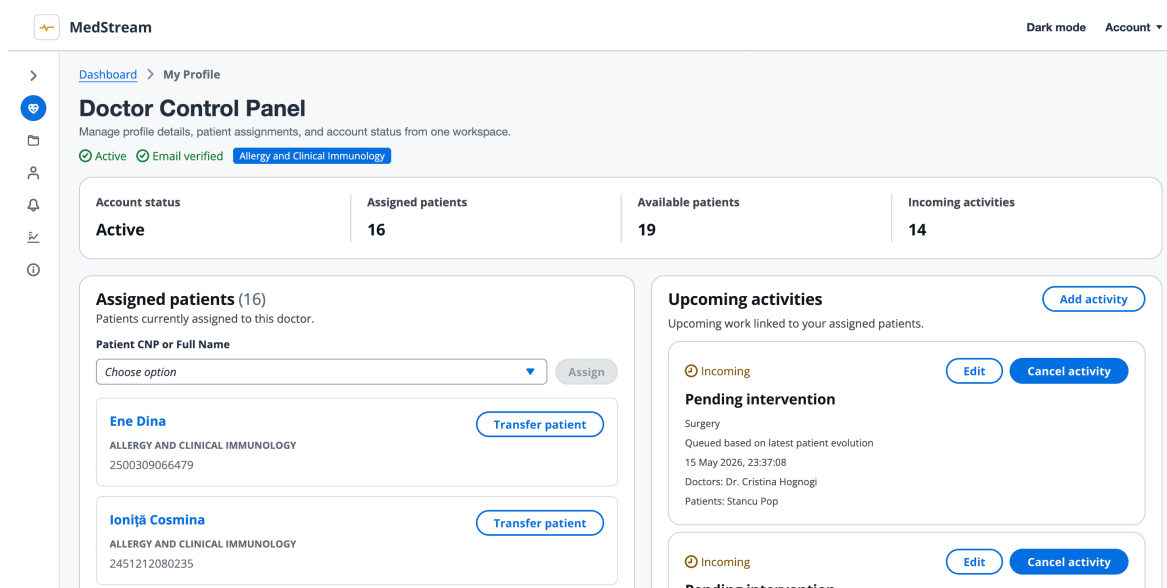


Figura 5.3: Pagina de profil a doctorului

- **Date afișate:** nume, prenume, email, telefon, specializare, număr de licență și data nașterii.
- **Editare profil:** doctorul poate actualiza datele de profil, iar schimbarea email-ului este tratată separat deoarece necesită reverificare.

- **Pacienți asignați:** doctorul poate vedea lista pacienților asociați și poate asigura sau elimina pacienți.
- **Activități medicale:** doctorul poate adăuga, edita sau anula activități medicale asociate pacienților.
- **Ștergere cont:** dezactivarea contului este permisă doar dacă sunt respectate regulile aplicației, de exemplu doctorul nu trebuie să fie singurul activ în departament și nu trebuie să aibă activități medicale în status `incoming`.

5.3.1.2 Managementul pacienților

Managementul pacienților susține fluxul clinic principal al aplicației. Doctorul poate adăuga pacienți, poate consulta profilul acestora, poate edita datele, poate gestiona istoricul clinic și poate urmări informații legate de internare, externare, readmitere și tratamente.

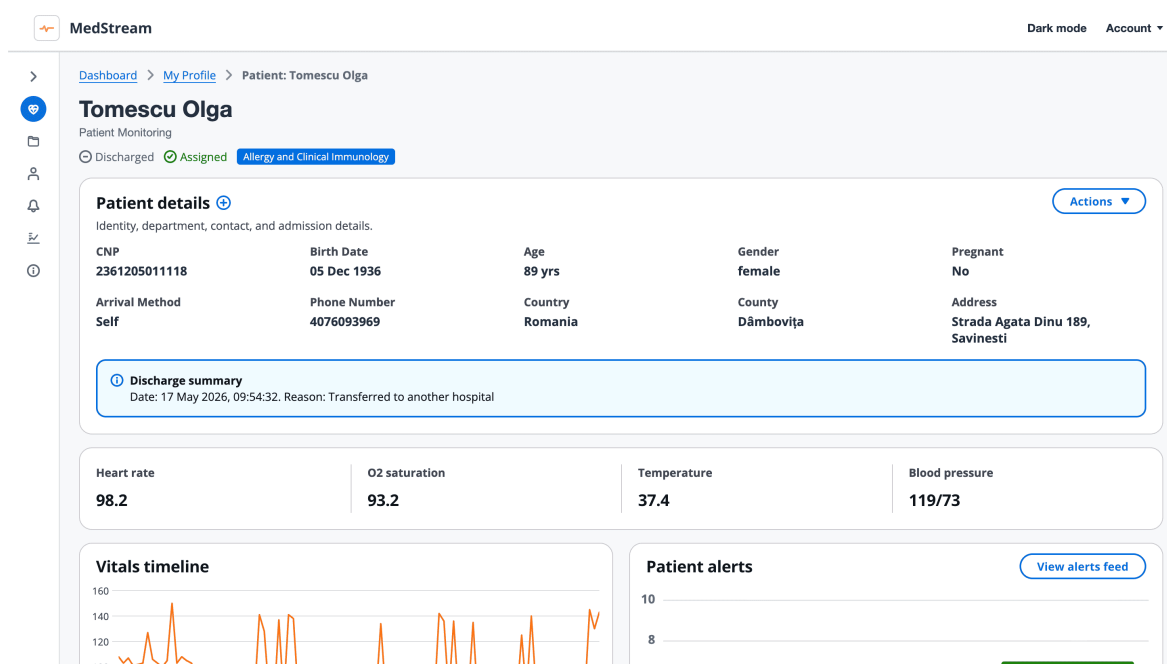


Figura 5.4: Profilul pacientului cu informații generale

- **Pagini implicate:** `/patients/new`, `/patient/:id`, `/departments/:name`, `/patients/:id/medical-history`, `/patients/:id/admission-history` și `/patients/:id/analysis`.
- **Date de identificare:** nume, prenume, CNP, data nașterii, vârstă și gen.
- **Date de contact și adresă:** telefon, stradă, număr, apartament, oraș, județ, cod poștal și țară.

- **Date clinice generale:** departament, metodă de sosire, stare internare, motiv externare și doctori asignați.
- **Date medicale asociate:** diagnostice, condiții medicale, alergii, medicație, activități medicale și istoric de internări.

Adăugarea unui pacient se realizează printr-un formular dedicat. Datele introduse sunt trimise către backend, validate și salvate în baza de date. După creare, pacientul poate fi asociat cu doctorul curent și poate apărea în lista departamentului corespunzător.

- **Creare pacient:** utilizatorul completează formularul de adăugare, iar backend-ul creează înregistrarea pacientului.
- **Asociere doctor:** pacientul poate fi asociat cu doctorul curent sau cu alți doctori disponibili.
- **Afișare în departament:** pacientul apare în pagina departamentului corespunzător, în funcție de datele selectate la creare sau transfer.

Profilul pacientului centralizează informațiile administrative și clinice. Această pagină funcționează ca punct principal de acces către istoricul clinic, istoricul internărilor și analiza tratamentului.

- **Informații afișate:** date personale, departament, stare internare, doctori asociați, valori vitale recente și alerte.
- **Acțiuni disponibile:** editare pacient, transfer, externare, readmitere și acces către paginile clinice.
- **Legătura cu alertele:** alertele generate pentru pacient sunt afișate în profil și pot fi analizate ulterior în contextul tratamentelor.

Aplicația tratează separat pacienții internați și pacienții externați. Această separare este importantă deoarece anumite acțiuni sunt disponibile doar pentru pacienții activi, iar readmiterea are sens doar pentru pacienții deja externați.

- **Pacient internat:** poate avea valori vitale noi, alerte, activități medicale și modificări ale datelor clinice.
- **Pacient externat:** este marcat vizual în interfață și păstrează motivul și data externării.
- **Readmitere:** poate fi realizată doar pentru pacienții externați, creând un nou episod de monitorizare.

- **Externare:** include reguli de validare, astfel încât pacientul să nu fie externat în condiții inconsistente cu starea sa recentă.

Istoricul clinic include diagnostice, condiții medicale, alergii și medicație. Aceste informații oferă context pentru interpretarea alertelor și pentru analiza tratamentelor.

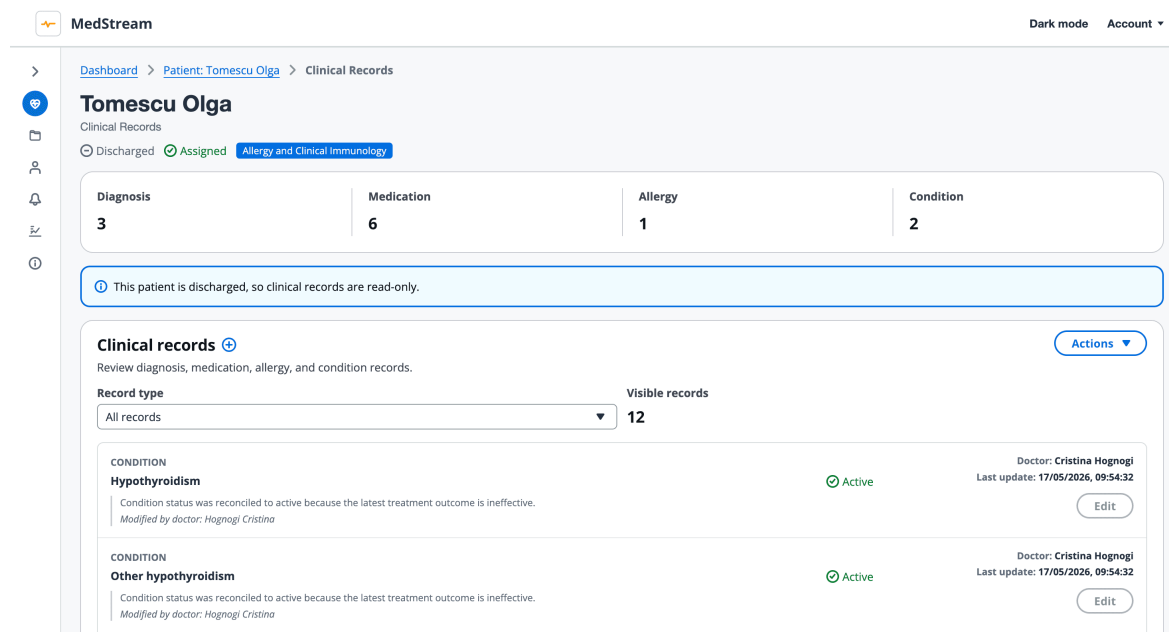


Figura 5.5: Pagina de istoric clinic, cu diagnostice, alergii, condiții medicale

- **Diagnostic:** permit păstrarea problemelor medicale asociate pacientului.
- **Condiții medicale:** descriu starea pacientului și evoluția sa.
- **Alergii:** includ informații despre severitate și riscuri relevante.
- **Medicație:** conține tratamentele, dozajul, frecvența și observațiile asociate.
- **Rol în aplicație:** istoricul clinic nu este direct streaming sau batch, dar oferă realism scenariului și date relevante pentru analiza tratamentului.

Managementul pacienților nu se limitează la operații CRUD simple, deoarece este conectat cu sistemul de alerte, monitorizarea valorilor vitale și analiza tratamentelor. Valorile generate pentru un pacient pot produce alerte, alertele pot influența interpretarea stării pacientului, iar datele istorice pot fi folosite ulterior pentru sumarul post-externare și pentru analiza retrospectivă.

5.3.2 Funcționalități generate de sistem

Funcționalitățile generate de sistem sunt procese automate care rulează fără intervenția directă a utilizatorului. Acestea includ generarea valorilor vitale de către simulator, publicarea evenimentelor în Kafka, consumul evenimentelor de către backend, actualizarea metricilor streaming, generarea alertelor, schimbarea stării pacientului, transferurile sau externările automate în scenarii simulate și rularea periodică a joburilor batch.

Aceste funcționalități susțin obiectivul aplicației, deoarece creează un flux continuu de date și permit compararea procesării streaming cu procesarea batch.

Streaming-ul reacționează la evenimente recente, iar batch-ul recalculează periodic snapshot-uri agregate pentru metrici, insight-uri și sumarul post-externare.

5.3.2.1 Dashboard și alerte

Dashboard-ul este pagina centrală a aplicației și oferă o vedere generală asupra stării sistemului. Acesta afișează numărul de pacienți activi, numărul de departamente, numărul de doctori și totalul alertelor urmărite. De asemenea, include un grafic live pentru valori vitale, precum puls, oxigen, temperatură și tensiune arterială.

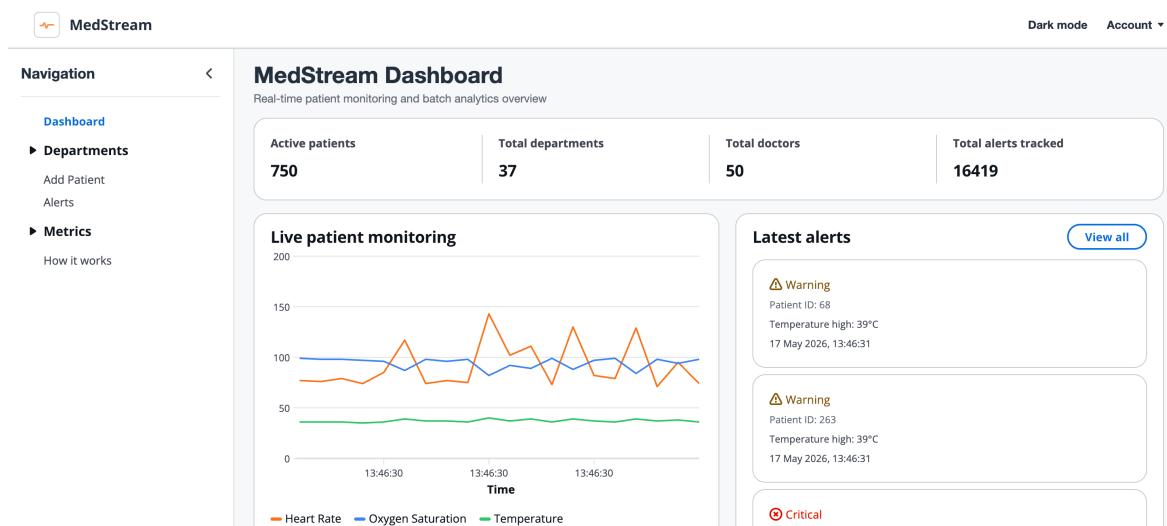


Figura 5.6: Pagina de start a aplicației

Dashboard-ul include și un panou cu alerte recente de severitate ridicată sau critică. Această zonă este utilă pentru supraveghere rapidă, deoarece permite doctorului să observe imediat pacienții care pot necesita atenție. Tehnic, dashboard-ul combină date operaționale curente cu unele informații agregate, oferind o perspectivă mixtă între monitorizare live și analiză generală.

Pagina de alerte afișează evenimentele generate atunci când valorile vitale depășesc limitele stabilite. Alerte pot fi filtrate după severitate, pacient sau ordine temporală, iar actualizările live sunt primite prin WebSocket. Această funcționalitate reprezintă una dintre cele mai clare zone de procesare streaming, deoarece alertele sunt generate incremental, pe măsură ce apar noi valori vitale.

5.3.2.2 Metrici streaming

Pagina de metrici streaming urmărește procesarea live a evenimentelor. Aceasta afișează indicatori precum `alerts per second`, `alerts per minute`, alerte noi de la ultimul update, fereastra curentă de alerte, media valorilor vitale și timpul de execuție. Pagina se actualizează frecvent și reflectă comportamentul recent al sistemului.

Graficele din această pagină pot include evoluția alertelor pe minut și tendințe pentru valorile vitale, cum ar fi pulsul, oxigenul și temperatura. Aceste vizualizări sunt utile pentru observarea activității curente și pentru demonstrarea latenței reduse a procesării streaming.

5.3.2.3 Metrici batch

Pagina de metrici batch urmărește analiza periodică a datelor acumulate. Utilizatorul poate configura programarea batch-ului la un anumit interval, poate porni manual procesarea prin `Run Batch Now` și poate urmări statusul execuției, etapa curentă, ultima rulare și timpul până la următoarea rulare.

Batch-ul agregă date pe o fereastră de analiză și calculează medii pentru puls, oxigen, temperatură și tensiune, număr de alerte, pacienți activi, evenimente procesate, rată de alerte și latență medie batch. În plus, pagina poate afișa insight-uri despre pacienți pe departamente, diagnostice frecvente și eficiența tratamentelor.

O parte importantă a paginii este analiza eficienței tratamentelor. Pentru fiecare medicament, aplicația poate calcula câte tratamente au fost `Effective`, `Improving` sau `Ineffective`, precum și distribuția acestor rezultate. Această zonă arată cum datele individuale ale pacienților pot fi transformate în indicatori comparabili prin procesare batch.

5.3.2.4 Comparatie streaming și batch

Pagina de comparație afișează diferențele dintre cele două paradigme de procesare. Streaming-ul este prezentat ca procesare imediată, cu latență redusă și alertare rapidă, în timp ce batch-ul este prezentat ca procesare periodică, cu latență mai mare, dar cu rezultate agregate și mai stabile.

Pagina poate compara throughput-ul, latența, rata alertelor, numărul total de evenimente, numărul total de alerte, alertele pe minut și alertele per rulare batch. De asemenea, interfața poate include grafice comparative pentru throughput și latență, cu intervale selectabile precum 1 minut, 3 minute, 5 minute, 15 minute sau 1 oră. Această funcționalitate susține direct obiectivul lucrării, deoarece arată diferența dintre reacția rapidă și analiza periodică.

5.3.2.5 Analiza tratamentului

Pagina de analiză a tratamentului urmărește evoluția pacientului în raport cu tratamentele aplicate, valorile vitale și alertele generate în timp. Scopul acestei pagini este de a oferi o interpretare retrospectivă asupra modului în care starea pacientului se modifică după anumite intervenții medicale simulate. Rezultatele afișate nu reprezintă decizii medicale reale, ci concluzii demonstrative, calculate prin reguli explicite definite în aplicație.

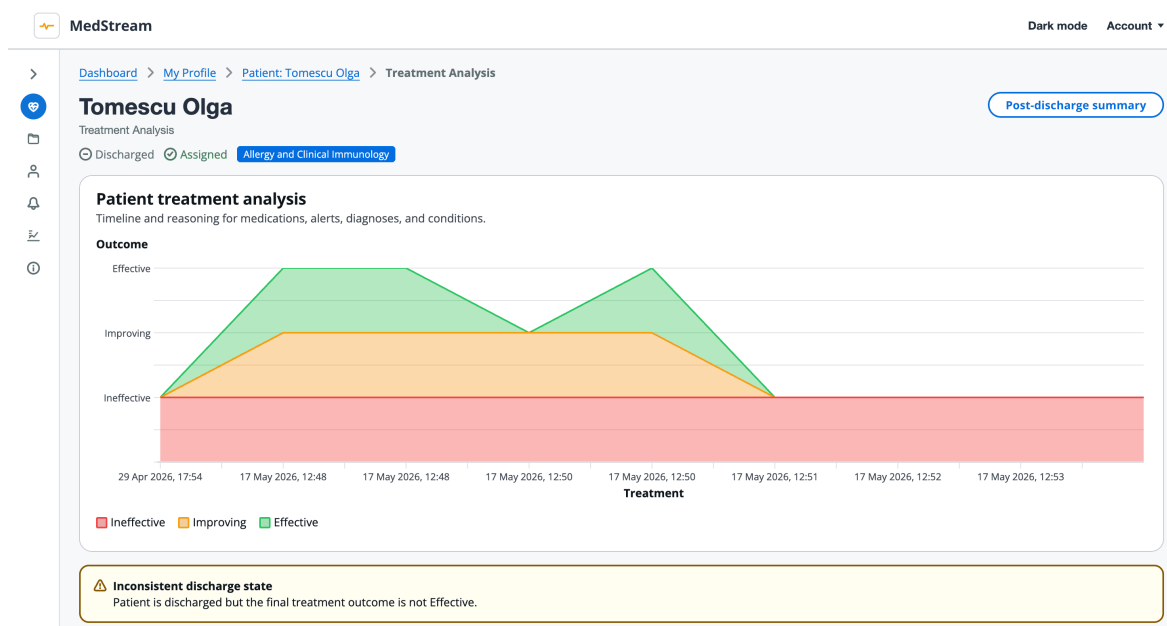


Figura 5.7: Pagina de analiză a tratamentului

Fluxul începe prin selectarea unui pacient și accesarea paginii de analiză. Backend-ul încarcă datele relevante despre pacient, tratamentele asociate, istoricul alertelor, diagnosticele, condițiile medicale și valorile necesare pentru interpretarea evoluției. Pe baza acestor date, aplicația construiește o imagine de ansamblu asupra stării pacientului și asupra reacției acestuia la tratamente.

- **Date despre pacient:** nume, departament, stare de internare și informații clinice generale.

- **Tratamente:** medicație administrată, dozaj, frecvență, observații și momentul aplicării.
- **Alerte:** evenimente generate pe baza valorilor vitale anormale, precum puls ridicat, oxigen scăzut sau temperatură anormală.
- **Context clinic:** diagnostice, condiții medicale și alte informații relevante din istoricul pacientului.
- **Rezultate calculate:** outcome-ul tratamentelor, sumarul alertelor și interpretarea evoluției pacientului.

Un element central al paginii este timeline-ul tratamentelor. Acesta afișează evoluția tratamentelor într-o formă vizuală, astfel încât utilizatorul să poată observa dacă rezultatul unui tratament este considerat *Effective*, *Improving* sau *Ineffective*. Aceste stări sunt determinate prin reguli explicite, în funcție de istoricul pacientului, de alertele apărute și de modificările observate în valorile vitale.

- **Effective:** tratamentul este interpretat ca având un rezultat favorabil, atunci când evoluția pacientului indică stabilizare sau lipsa unor alerte relevante după aplicare.
- **Improving:** tratamentul indică o evoluție parțial pozitivă, atunci când unele valori se îmbunătățesc, dar pacientul nu este complet stabil.
- **Ineffective:** tratamentul este interpretat ca neavând efectul dorit, atunci când alertele persistă sau valorile vitale rămân problematice.

Pe lângă timeline, pagina afișează și un sumar al celor mai recente alerte. Acest sumar ajută la înțelegerea contextului în care este evaluat tratamentul. De exemplu, dacă după administrarea unui tratament apar în continuare alerte critice, sistemul poate marca evoluția ca nefavorabilă. Dacă alertele scad sau valorile vitale se normalizează, rezultatul poate fi interpretat ca îmbunătățire sau eficiență. Această pagină nu este strict o pagină de streaming și nici strict o pagină de batch. Componenta streaming este prezentă prin legătura cu alertele recente și cu evenimentele generate pe baza valorilor vitale. În același timp, pagina are o componentă retrospectivă, deoarece analizează tratamentele pe baza datelor deja acumulate. Din acest motiv, analiza tratamentului funcționează ca o punte între partea operațională a aplicației și partea analitică.

Pașii principali ai fluxului de analiză sunt următorii:

- doctorul accesează profilul pacientului;

- din profil este deschisă pagina de analiză a tratamentului;
- frontend-ul solicită de la backend datele necesare pentru analiză;
- backend-ul returnează tratamentele, alertele, diagnosticele, condițiile și sumarul pacientului;
- interfața afișează timeline-ul tratamentelor și sumarul alertelor;
- sistemul marchează fiecare tratament cu un outcome calculat prin reguli;
- după externare, pagina poate afișa și un rezumat post-externare.

După externarea pacientului, aplicația poate afișa un rezumat post-externare. Acest overview are caracter retrospectiv și este apropiat de procesarea batch, deoarece sintetizează date istorice despre tratamente, alerte și evoluția pacientului. Prin această zonă, sistemul arată că batch-ul nu este folosit doar pentru metrice generale, ci și pentru generarea unor concluzii agregate la finalul unui episod clinic.

Rezumatul post-externare poate include informații precum statusul sumarului, interpretarea evoluției pacientului, distribuția tratamentelor și scoruri agregate. Un exemplu este *Treatment Response Score*, care sintetizează rezultatele tratamentelor într-o valoare mai ușor de interpretat. Această informație nu este folosită pentru decizii medicale reale, ci pentru a demonstra cum datele acumulate pot fi transformate într-un rezultat retrospectiv.

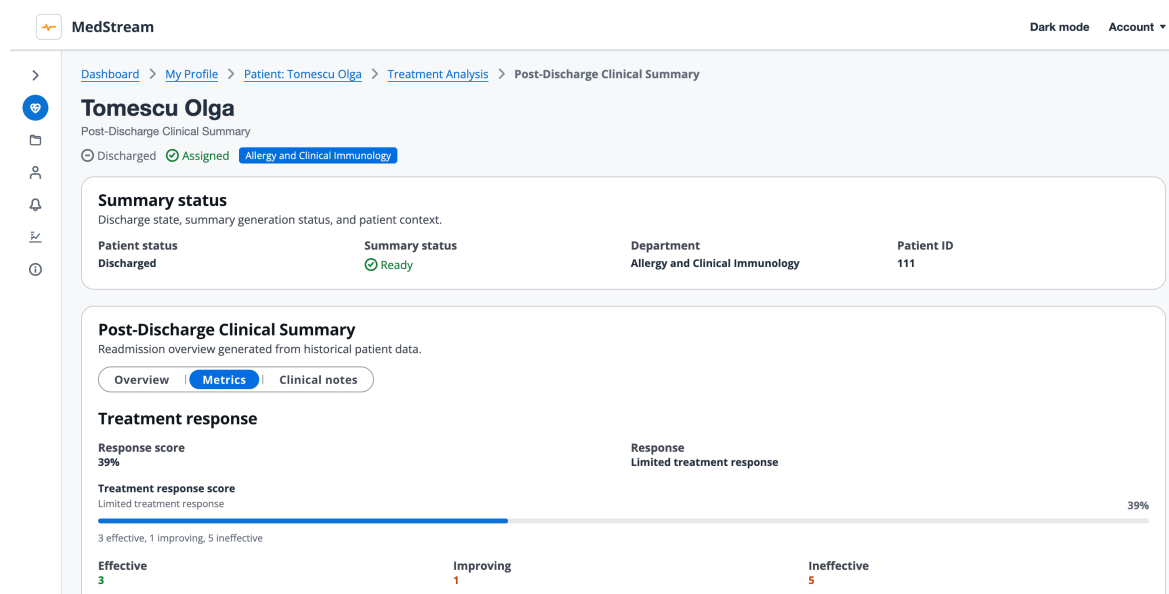


Figura 5.8: Rezumatul post-externare generat pe baza datelor istorice

- **Statusul sumarului:** poate indica dacă rezumatul este disponibil, în așteptare sau nu poate fi generat.
- **Treatment Response Score:** exprimă sintetic răspunsul pacientului la tratamentele simulate.

- **Distribuția outcome-urilor:** arată câte tratamente au fost eficiente, în îmbunătățire sau ineficiente.
- **Interpretare retrospectivă:** oferă o concluzie generală pe baza datelor istorice ale pacientului.

Prin această funcționalitate, aplicația evidențiază modul în care evenimentele individuale, precum valorile vitale și alertele, pot fi corelate cu informații clinice și transformate într-o analiză mai amplă. Astfel, pagina Treatment Analysis susține obiectivul aplicației MedStream, deoarece combină informații recente cu interpretări retrospective și arată complementaritatea dintre procesarea streaming și procesarea batch într-un scenariu clinic simulat.

Funcționalitățile și rezultatele analizate arată că MedStream reușește să evidențieze diferența dintre cele două modele de procesare. Actualizările live, alertele și valorile recente ilustrează rolul streaming-ului, în timp ce metricile agregate, snapshot-urile și analiza tratamentelor ilustrează rolul procesării batch.

Observațiile obținute permit formularea concluziilor generale ale lucrării, împreună cu limitele sistemului implementat și direcțiile posibile de extindere.

Capitolul 6

Concluzii

În cadrul acestei lucrări a fost proiectată și implementată aplicația MedStream, un sistem demonstrativ construit pentru compararea procesării streaming cu procesarea batch. Aplicația folosește un scenariu de monitorizare clinică simulată, deoarece acest tip de context permite evidențierea clară a celor două nevoi principale urmărite în lucrare: reacția rapidă la evenimente recente și analiza periodică a datelor acumulate.

Sistemul nu a fost gândit ca o aplicație medicală reală, ci ca un instrument educativ și experimental. Prin folosirea unor date clinice sintetice, aplicația permite observarea modului în care valorile vitale pot genera alerte în timp aproape real, dar și a modului în care aceleași date pot fi folosite ulterior pentru statistici, metrice și interpretări retrospective.

Lucrarea a pornit de la prezentarea conceptelor teoretice necesare pentru înțelegerea celor două modele de procesare, apoi a descris arhitectura sistemului și implementarea aplicației. MedStream integrează mai multe componente tehnice, printre care React pentru interfața web, FastAPI pentru backend, PostgreSQL pentru persistența datelor, Kafka pentru fluxul de evenimente, WebSocket pentru actualizări live și mecanisme batch pentru calcule periodice. Această separare a componentelor a permis construirea unui sistem în care fluxul live și fluxul periodic pot fi urmărite separat, dar în cadrul aceluiași scenariu.

Un rezultat important al lucrării este faptul că aplicația depășește nivelul unui sistem CRUD simplu. MedStream include un simulator de date, un flux de evenimente, un consumator pentru procesarea alertelor, actualizări live în interfață, calcule batch, pagini de metrice și o analiză rule-based a tratamentelor. Prin aceste componente, aplicația oferă un cadru practic pentru observarea diferenței dintre latența redusă a procesării streaming și caracterul mai stabil al rezultatelor obținute prin batch.

În urma implementării, s-a observat că cele două abordări nu trebuie privite ca soluții opuse, ci ca modele potrivite pentru nevoi diferite. Streaming-ul este util

atunci când sistemul trebuie să reacționeze rapid, de exemplu la apariția unei valori vitale anormale. Batch-ul este mai potrivit atunci când scopul este analiza unor date deja acumulate, cum ar fi calcularea unor medii, agregarea informațiilor sau obținerea unei imagini de ansamblu asupra evoluției pacientului ori a activității din sistem.

Sistemul implementat are și limitări. Datele folosite sunt simulate, iar regulile de alertare și analiză sunt definite explicit în cod. Aplicația nu include validare clinică reală și nu folosește modele avansate de predicție sau machine learning. De asemenea, pentru utilizarea într-un mediu real ar fi necesare componente suplimentare, precum mecanisme mai stricte de securitate, audit, migrații versionate ale bazei de date, testare pe volume mari de date și reguli medicale validate de specialiști.

Ca direcții viitoare de dezvoltare, aplicația ar putea fi extinsă prin folosirea unor surse de date mai realiste, prin îmbunătățirea simulatorului și prin adăugarea unor scenarii clinice mai variate. O extensie utilă ar fi introducerea unui mod demonstrativ dedicat comparației directe dintre streaming și batch, în care același set de evenimente să poată fi urmărit în paralel prin ambele fluxuri de procesare. Acest mod ar putea permite modificarea intervalului batch, ajustarea vitezei simulatorului și afișarea diferențelor de latență, actualizare și agregare într-o formă vizuală mai clară.

În concluzie, MedStream arată cum procesarea streaming și procesarea batch pot fi combinate într-un sistem modern de date. Streaming-ul oferă reacție rapidă și este potrivit pentru monitorizare live și alerte, în timp ce batch-ul oferă rezultate agregate și este potrivit pentru analize periodice. Prin integrarea ambelor abordări într-o singură aplicație, lucrarea oferă o demonstrație practică a modului în care aceste modele se completează în funcție de cerințele sistemului.

Bibliografie

- [ABC⁺15] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, et al. The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.
- [Ama26a] Amazon Web Services. Aws glue, 2026.
- [Ama26b] Amazon Web Services. Aws glue concepts, 2026.
- [Ama26c] Amazon Web Services. Building visual etl jobs, 2026.
- [Ama26d] Amazon Web Services. What is a modern data streaming architecture?, 2026.
- [Ama26e] Amazon Web Services. What is batch processing?, 2026.
- [Apa26a] Apache Software Foundation. Apache beam programming guide, 2026.
- [Apa26b] Apache Software Foundation. Apache kafka documentation, 2026.
- [Apa26c] Apache Software Foundation. Cluster mode overview, 2026.
- [Apa26d] Apache Software Foundation. Kafka streams architecture, 2026.
- [Apa26e] Apache Software Foundation. Overview, 2026.
- [APS26] APScheduler. Apscheduler documentation, 2026.
- [CKE⁺15] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 38(4):28–38, 2015.
- [Clo26] Cloudscape Design System. Cloudscape design system, 2026.

- [Con26] Confluent. Stream processing concepts in confluent cloud for apache flink, 2026.
- [Dat26] Databricks. Batch vs. streaming data processing in databricks, 2026.
- [Dee24] M. Deebhan. Streaming vs batch processing: Why real-time matters more than ever, 2024.
- [DG08] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.
- [Doc26] Docker. What is docker?, 2026.
- [Fas26a] FastAPI. Cors cross-origin resource sharing, 2026.
- [Fas26b] FastAPI. Fastapi, 2026.
- [IBM26] IBM. What is three-tier architecture?, 2026.
- [iDe25] iDempiere. Understanding rest api and its importance in idempiere development, 2025.
- [K⁺19] Martin Kleppmann et al. Designing data-intensive applications, 2019.
- [Kre14] Jay Kreps. Questioning the lambda architecture, 2014.
- [Kum23] Rajesh Kumar. Product design: Three-tier architecture, 2023.
- [Moz26a] Mozilla Developer Network. Overview of http, 2026.
- [Moz26b] Mozilla Developer Network. Spa single-page application, 2026.
- [Moz26c] Mozilla Developer Network. Websocket api, 2026.
- [Ott25] Neri Van Otten. Real-time vs batch processing made simple: What is the difference?, 2025.
- [PMdS⁺22] Antonio Iyda Paganelli, Abel Gonzalez Mondejar, Abner Cardoso da Silva, Greis Silva-Calpa, Mateus F Teixeira, Felipe Carvalho, Alberto Raposo, and Markus Endler. Real-time data analysis in health monitoring systems: A comprehensive systematic literature review. *Journal of Biomedical Informatics*, 127:104009, 2022.
- [Pos26] PostgreSQL Global Development Group. Architectural fundamentals, 2026.

- [PPP26] Massimo Pacella, Antonio Papa, and Gabriele Papadia. Towards intelligent fused filament fabrication: Computational verification of a monitoring and early-warning framework for instability mitigation. *Applied Sciences*, 16(1):459, 2026.
- [PPPF25] Massimo Pacella, Antonio Papa, Gabriele Papadia, and Emiliano Fedeli. A scalable framework for sensor data ingestion and real-time processing in cloud manufacturing. *Algorithms*, 18(1):22, 2025.
- [Pyt26] Python Software Foundation. venv — creation of virtual environments, 2026.
- [Ram19] Sivasubramanian Ramani. Orchestrating an application process with aws batch using aws cloudformation, 2019.
- [Rea26a] React. Built-in react hooks, 2026.
- [Rea26b] React. Quick start, 2026.
- [Rea26c] React. React documentation, 2026.
- [Rea26d] React. Understanding your ui as a tree, 2026.
- [Rec26] Recharts. Recharts, 2026.
- [Sca24] Scaler Topics. Postgresql architecture, 2024.
- [SÇZ05] Michael Stonebraker, Uğur Çetintemel, and Stan Zdonik. The 8 requirements of real-time stream processing. *ACM Sigmod Record*, 34(4):42–47, 2005.
- [SQL26] SQLAlchemy. Mapping classes with declarative, 2026.
- [Tex14] Textractor. Diagram of lambda architecture generic, 2014.
- [WM15] James Warren and Nathan Marz. *Big Data: Principles and best practices of scalable realtime data systems*. Simon and Schuster, 2015.
- [ZDL⁺13] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the twenty-fourth ACM symposium on operating systems principles*, pages 423–438, 2013.